



**Kauno technologijos universitetas**

Informatikos fakultetas

# **Laisvalaikio įrangos dalinimosi platforma**

Baigiamasis bakalauro studijų projektas

---

**Redas Niuklys**

Projekto autorius

**asist. dr. Žitkus Voldemaras**

Vadovas

---

**Kaunas, 2026**



**Kauno technologijos universitetas**

Informatikos fakultetas

## **Laisvalaikio įrangos dalinimosi platforma**

Baigiamasis bakalauro studijų projektas

Programų sistemos (612I30002)

---

**Redas Niuklys**

Projekto autorius

**asist. dr. Žitkus Voldemaras**

Vadovas

**doc. pr. Mažutienė Rasa**

Recenzentas

---

**Kaunas, 2026**



**Kauno technologijos universitetas**

Informatikos fakultetas

Redas Niuklys

## **Laisvalaikio įrangos dalinimosi platforma**

### **Akademinių sąžiningumo deklaracija**

Patvirtinu, kad:

1. baigiamąjį projektą parengiau savarankiškai ir sąžiningai, nepažeisdama(s) kitų asmenų autoriaus ar kitų teisių, laikydamasi(s) Lietuvos Respublikos autorių teisių ir gretutinių teisių įstatymo nuostatų, Kauno technologijos universiteto (toliau – Universitetas) intelektinės nuosavybės valdymo ir perdavimo nuostatų bei Universiteto akademinės etikos kodekse nustatytų etikos reikalavimų;
2. baigiamajame projekte visi pateikti duomenys ir tyrimų rezultatai yra teisingi ir gauti teisėtai, nei viena šio projekto dalis nėra plagijuota nuo jokių spausdintinių ar elektroninių šaltinių, visos baigiamojo projekto tekste pateiktos citatos ir nuorodos yra nurodytos literatūros sąrašė;
3. įstatymų nenumatytų piniginių sumų už baigiamąjį projektą ar jo dalis niekam nesu mokėjęs (-usi);
4. suprantu, kad išaiškėjus nesąžiningumo ar kitų asmenų teisių pažeidimo faktui, man bus taikomos akademinės nuobaudos pagal Universitete galiojančią tvarką ir būsiu pašalinta(s) iš Universiteto, o baigiamasis projektas gali būti pateiktas Akademinės etikos ir procedūrų kontrolieriaus tarnybai nagrinėjant galimą akademinės etikos pažeidimą.

Redas Niuklys

*Patvirtinta elektroniniu būdu*

Niuklys, Redas. Laisvalaikio įrangos dalinimosi platforma. Bakalauro studijų baigiamasis projektas / vadovas asist. dr. Žitkus Voldemaras; Kauno technologijos universitetas, Informatikos fakultetas.

Studijų kryptis ir sritis (studijų krypčių grupė): Informatikos mokslai, Programų sistemos.

Reikšminiai žodžiai: programėlė, dalinimasis, rezervacijos, bendruomenės

Kaunas, 2026. 51 p.

## **Santrauka**

Darbe pristatoma laisvalaikio prekių dalinimosi platforma, skirta privačios draugų grupėms ar bendros įmonės darbuotojams dalintis retai naudojamais daiktais. Kadangi esami sprendimai arba yra neprieinami Lietuvoje, arba išlieka paprasčiausias būdas – trumposios žinutės skirtingose pokalbių platformose. Viskas vyksta žodiniais pažadais be jokio atsekamumo. Kuriama sistema išsprędžią šią nežinojimo problemą rezervacijos sistema, žemėlapiu aktyviems skelbimams ir įrangų sąrašais.

Sistemos projektavimo pradžioje atliktos konkurentų analizės metu išsiaiškinta, jog Lietuvoje tokia paslauga nėra paplitusi. Didesnis susidomėjimas tokiems sprendimams yra Vakarų Europoje (Belgija, Vokietija). Lietuvoje pagrindinė alternatyva – nuoma. Nėra dedikuotų portalų išnaudoti įrangos galerijų privalumus iš nuomos svetainių privatiems dalinimosi ratams. Lieka tik žodiniai sutarimai.

Sistema realizuota naudojantis *Typescript* ir *C#* programavimo kalbomis ir *React Native* ir *ASP.NET* karkasais atitinkamoms kalboms. Programinio kodo serverinė dalis padengta integraciniais testais, kurie leido numatyti galimas trikdžių vietas vykstant skirtingiems scenarijais. Sprendimas patalpintas AWS debesijos paslaugų tiekėjo serveryje, naudotojo sąsajos dalis dalinamasi Android operacinės sistemos „Android“ paketo failu (.apk)

Niuklys, Redas. Leisure Equipment Sharing Platform. Bachelor's Final Degree Project / supervisor asst. dr. Žitkus Voldemaras; Informatics Faculty, Kaunas University of Technology.

Study field and area (study field group): Computer Sciences, Software Systems.

Keywords: app, sharing, rezervations, communities

Kaunas, 2026. 51 pages.

### **Summary**

The thesis presents a leisure goods sharing platform designed for private groups of friends or employees within the same company to share rarely used items. Existing solutions are either unavailable in Lithuania, or the most common approach remains short messages across various chat platforms. Everything is handled through verbal promises without any traceability. The developed system solves this uncertainty problem through a reservation system, a map for active listings, and equipment lists.

During the initial system design phase, competitor analysis revealed that such services are not widespread in Lithuania. Greater interest in these types of solutions exists in Western Europe (Belgium, Germany). In Lithuania, the main alternative is rental services. There are no dedicated platforms that leverage the advantages of equipment galleries from rental websites for private sharing circles. As a result, only verbal agreements remain.

The system is implemented using the *TypeScript* and *C#* programming languages, along with the *React Native* and *ASP.NET* frameworks respectively. The server-side codebase is covered by integration tests, which helped identify potential failure points among different scenarios. The solution is hosted on servers provided by the AWS cloud services platform, while the user interface is distributed as an Android application package (.apk) file for the Android operating system.

## Turinys

|                                                                                         |           |
|-----------------------------------------------------------------------------------------|-----------|
| <b>Lentelių sąrašas .....</b>                                                           | <b>7</b>  |
| <b>Paveikslų sąrašas .....</b>                                                          | <b>8</b>  |
| <b>Santrumpų ir terminų sąrašas .....</b>                                               | <b>9</b>  |
| <b>Įvadas.....</b>                                                                      | <b>10</b> |
| <b>1. Analizė .....</b>                                                                 | <b>11</b> |
| 1.1. Techninis pasiūlymas .....                                                         | 11        |
| 1.1.1. Sistemos apibrėžimas .....                                                       | 11        |
| 1.1.2. Bendras veiklos tikslas .....                                                    | 11        |
| 1.1.3. Sistemos pagrįstumas .....                                                       | 11        |
| 1.1.4. Egzistuojančių sprendimų analizė .....                                           | 11        |
| 1.1.5. Prototipai ir pagalbinių informacija .....                                       | 13        |
| 1.2. Techninių galimybių analizė .....                                                  | 13        |
| <b>2. Projektas.....</b>                                                                | <b>15</b> |
| 2.1. Reikalavimų specifikacija .....                                                    | 15        |
| 2.1.1. Sistemos funkcijos.....                                                          | 15        |
| 2.1.2. Realizacijai keliami reikalavimai .....                                          | 18        |
| 2.1.3. Techninė specifikacija .....                                                     | 18        |
| 2.2. Projektavimo metodai.....                                                          | 18        |
| 2.2.1. Projektavimo valdymas ir eiga .....                                              | 18        |
| 2.2.2. Projektavimo technologija.....                                                   | 19        |
| 2.2.3. Programavimo kalbos, derinimo, automatizavimo priemonės, operacinė sistemos..... | 19        |
| 2.3. Sistemos projektas .....                                                           | 19        |
| 2.3.1. Statinis sistemos vaizdas .....                                                  | 19        |
| 2.3.2. Dinaminis sistemos vaizdas.....                                                  | 24        |
| <b>3. Testavimas.....</b>                                                               | <b>30</b> |
| 3.1. Testavimo planas .....                                                             | 30        |
| 3.2. Testavimo kriterijai .....                                                         | 30        |
| 3.3. Integracinis testavimas .....                                                      | 30        |
| <b>4. Dokumentacija naudotojui .....</b>                                                | <b>33</b> |
| 4.1. Apibendrintas sistemos galimybių aprašymas.....                                    | 33        |
| 4.2. Naudotojo vadovas .....                                                            | 33        |
| 4.3. Diegimo vadovas .....                                                              | 47        |
| 4.4. Administravimo vadovas.....                                                        | 48        |
| <b>5. DI naudojimo savirefleksija.....</b>                                              | <b>49</b> |
| <b>Rezultatai ir išvados .....</b>                                                      | <b>50</b> |
| <b>Literatūros sąrašas .....</b>                                                        | <b>51</b> |

## **Lentelių sąrašas**

|                                                                |    |
|----------------------------------------------------------------|----|
| lentelė 1 Sprendimų palyginimas .....                          | 13 |
| lentelė 2 PA „Redaguoti įrangą“ specifikacija .....            | 17 |
| lentelė 3 PA „Keisti rezervacijos statusą“ specifikacija ..... | 17 |
| lentelė 4 PA „Sukurti vietovę“ specifikacija .....             | 17 |
| lentelė 5 PA „Rašyti žinutę“ specifikacija.....                | 18 |
| lentelė 6 Programinė įranga.....                               | 19 |

## **Paveikslų sąrašas**

|                |                                                                               |    |
|----------------|-------------------------------------------------------------------------------|----|
| <b>pav. 1</b>  | „Stuffly“ galimų skolintis daiktų sąrašas .....                               | 12 |
| <b>pav. 2</b>  | „Peerby“ pagrindinis puslapis .....                                           | 13 |
| <b>pav. 3</b>  | Sistemos panaudojimo atvejų diagrama .....                                    | 16 |
| <b>pav. 4</b>  | Diegimo diagrama .....                                                        | 20 |
| <b>pav. 5</b>  | Paketų diagrama .....                                                         | 21 |
| <b>pav. 6</b>  | Duomenų bazės schema .....                                                    | 23 |
| <b>pav. 7</b>  | PA „Keisti rezervacijos statusą“ būsenų diagrama .....                        | 25 |
| <b>pav. 8</b>  | PA „Sukurti vietovę“ sekų diagrama.....                                       | 26 |
| <b>pav. 9</b>  | PA „Pridėti nuotraukas“ sekų diagrama .....                                   | 27 |
| <b>pav. 10</b> | PA „Pridėti įrangą“ sekų diagrama .....                                       | 28 |
| <b>pav. 11</b> | PA „Sukurti rezervaciją“ sekų diagrama.....                                   | 29 |
| <b>pav. 12</b> | Serverio dalies testų padengimas .....                                        | 31 |
| <b>pav. 13</b> | Testų vykdymo eiga .....                                                      | 32 |
| <b>pav. 14</b> | Prisijungimo langas .....                                                     | 34 |
| <b>pav. 15</b> | Pagrindinis langas – Mano įranga .....                                        | 34 |
| <b>pav. 16</b> | Pagrindinis langas tamsiąja tema .....                                        | 35 |
| <b>pav. 17</b> | Įrangos pridėjimas/redagavimas .....                                          | 36 |
| <b>pav. 18</b> | Naujos vietovės pridėjimas .....                                              | 37 |
| <b>pav. 19</b> | Įrangos informacijos langas 1/2 .....                                         | 38 |
| <b>pav. 20</b> | Įrangos informacijos langas 2/2 Rezervacijos, Atsiliepimai, Redagavimas ..... | 39 |
| <b>pav. 21</b> | Rezervacijų sąrašas.....                                                      | 40 |
| <b>pav. 22</b> | Nauja rezervacija.....                                                        | 41 |
| <b>pav. 23</b> | Atsiliepimų modalinis langas .....                                            | 42 |
| <b>pav. 24</b> | Įrangų žemėlapis su atstumais .....                                           | 43 |
| <b>pav. 25</b> | Profilio langas .....                                                         | 44 |
| <b>pav. 26</b> | Nustatymų langas su kalbos pasirinkimu .....                                  | 44 |
| <b>pav. 27</b> | Administratoriaus sąsaja .....                                                | 45 |
| <b>pav. 28</b> | Pokalbių sąrašas .....                                                        | 46 |
| <b>pav. 29</b> | Pokalbio langas .....                                                         | 47 |



## **Santrumpų ir terminų sąrašas**

### **Santrumpos:**

ORM – Object–relational mapper

OAuth – Open Authorization

AWS – Amazon Web Services

### **Terminai:**

Modalinis dialogo langas – „Operacinės sistemos arba kitos programos dialogo langas, atveriamas priekiniame plane ir neleidžiantis naudotojui padaryti aktyviais kitų tos pačios programos langų, kol jame nebus surinkti reikiami duomenys arba kol jis nebus užvertas.“

## Ivadas

Platforma buvo sukurta, siekiant susistemizuoti dalinimosi/skolinimosi procesą tarp pažįstamų ar draugų. Esami sistemizuoti sprendimai nėra pritaikyti Lietuvos rinkai, kur vis dar vyrauja tokio tipo laikinas daiktų turėjimas – nuomos principu iš viešųjų asmenų. Dalinimasis privačiuose ratuose vyksta žinučių pagalba, kas apriboja atsekamumą, matomumą ir bendrą patogumą. Reikalingos išankstinės žinios ar asmuo turi tokią įrangą, ar ja nori dalintis, ar jau nėra kam nors kitam paskolines. Tokio tipo sistemizuotos platformos Lietuvos rinkoje nėra.

Tikslas – sukurti platformą, kurioje naudotojai gali paprastai, patogiai ir susistemintai dalintis turima laisvalaikio įranga. Tam įvykdyti buvo iškelti šie uždaviniai:

1. Atlikti konkurentų analizę panašioms platformoms
2. Suprojektuoti ir sumodeliuoti platformos struktūrą
3. Realizuoti suprojektuotą sistemą
4. Sistemą patalpinti debesijos paslaugų platformoje platesniam pasiekiamumui
5. Ištestuoti sukurta platformą ir parengti naudotojo vadovą sklandžiam naudojimui

Darbo dokumentas susideda iš 5 skyrių: analizė, projektavimas, testavimas, dokumentavimas bei DI panaudojimas. Pirmoje dalyje atliekta analizė konkurentų/analogiškų sprendimų prieinamumas, skirtumai su norima kurti sistema. Projektavime – pateikta funkciniai, nefunkciniai reikalavimai, atvaizduotos diagramos, nusakančios statinį bei dinaminį sistemos vaizdą. Testavime – įvertinamas sistemos serverinės dalies padengimas skirtingais integracinių testų atvejais. Dokumentavime pateikiamas vadovas ir naudotojui su pavyzdiniu atveju naudojimuisi bei administratoriaus atmintinė kaip reikalinga sudiegti serverinę dalį į AWS serverį. Priešpaskutiniame skyrelyje nurodyta kaip projekte buvo išnaudotos Dirbtinio intelekto galimybės, siekiant sumažinti krūvį ir optimizuoti darbų eigą. Pabaigoje pateikiamos išvados ir rezultatai gauti iš sukurtos sistemos.

Sukurtos sistemos apimtis:

- Naudotojo sąsajos programinio kodo eilučių kiekis: 5359
- Serverinės dalies programinio kodo eilučių kiekis: 5013
- Kodo padengimas integraciniais testais serverio dalyje: 78%
- Projekte parašytų testų kiekis : 291
- Darbo valandos, skirtos projektui ~300

## **1. Analizė**

Šiame skyriuje pateikiama kuriamos sistemos analizė. Atsižvelgiama į tikslą, pagrįstumą bei konkurentų analizę. Taip pat nurodomos pasirinktos naudoti technologijos pagal keliamus poreikius.

### **1.1. Techninis pasiūlymas**

#### **1.1.1. Sistemos apibrėžimas**

Kuriama platforma – tai susistemizuota laisvalaikio įrangos dalinimosi platforma, kuri pakeičia įprastus žodinius, žinučių susitarimus pakeis rezervacijų sistema. Sistema leis pridėti savo turimas įrangas kartus su jos nuotraukomis, tipu, aprašu bei lokacija. Kiti naudotojai šias įrangas gali pateikti rezervacijai, kurias tvirtina savininkas. Sklandžiam bendravimui platformoje taip pat yra susirašinėjimo funkcionalumas, kuris yra integruotas į rezervavimo procesą. Išnaudojant pateikiamą informaciją kitiems vartotojams – matomas bendras vaizdas ar įranga yra populiari, kokie yra jos atsiliepimai bei užimtumas norimomis skolintis dienomis.

#### **1.1.2. Bendras veiklos tikslas**

Veiklos tikslas – siūlyti galimą alternatyvą įrangos dalinimuisi. Išnaudojant platformą yra atnešama daugiau matomumo galimiems pasiūlymams, gal net naujų draugysčių užmesgimui, jeigu sistema sudiegiamą įmonės mastu. Platforma suteikia paprastesnę sekimą savo bei kitoms galimoms įrangoms/daiktams. Su esamais filtrais bei reitingais naudotojas galės lengviau išsirinkti iš didelio sąrašo įrangų ar siekti išbandyti naujas veiklas, neįvestuojant papildomų išlaidų bandymams.

#### **1.1.3. Sistemos pagrįstumas**

Platformos pagrįstumas atsiranda iš matomos terpės Lietuvos rinkoje tokio tipo platformoms. Yra egzistuojančios nuomos platformos iš viešųjų asmenų, bet draugų/įmonės mastu tokios platformos Lietuvoje nėra paplitusios. Egzistuoja sprendimų, kurtų kitoms šalims, tačiau Lietuvoje susistemizuotos dalinimosi platformos dabar neturime. Esami geriausi sprendimai yra „Facebook“ grupės ar asmeninės žinutės, tačiau šiems sprendimams trūksta rezervavimo/matomumo sprendimų. Rezervacija užtikrina prieinamumą ir tampa kertine atostogų planavimo dalimi.

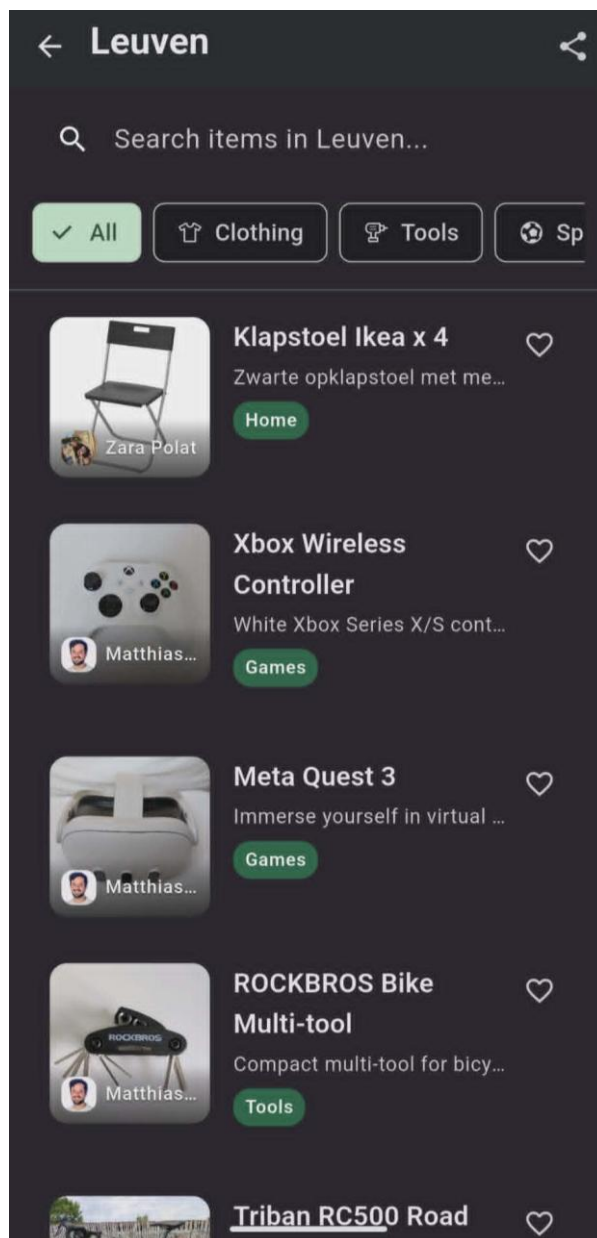
#### **1.1.4. Egzistuojančių sprendimų analizė**

Tokio tipo dalinimosi sistemų nėra daug. Kitos sistemos stengiasi apmokestinti turimą turtą – veikti kaip nuomos punktai. Jose atsiranda ir įmonės nuomojančios savo parduodamus daiktus. Atlikus detalesnę analizę atrinkta 2 egzistuojantys, tinkamiausi sprendimai palyginimo analizei: „Peerby“ bei „Stuffly“. Toliau bus pateikiama šių sprendimų trumpa analizė bei palyginimas su kuriu sprendimu.

Pradedame nuo „Stuffly“, kadangi jis turi galimybę dalintis privačiose grupėse. Aplikacija turi patogų filtravimą pagal kategorijas, (1) naudoja AI greitam daiktų sukūrimui iš nuotraukos.

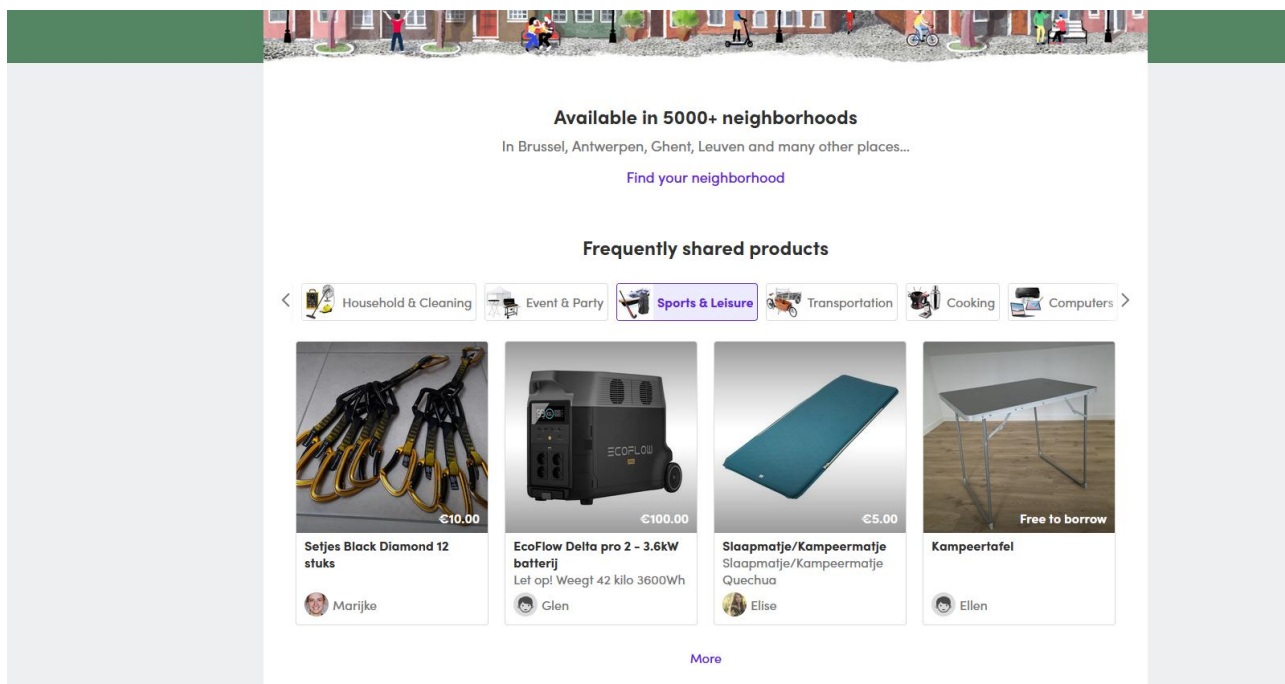
Visos rezervacijos yra kalendoriaus principu.

Vienas pastebėtas nepatogumas – orientacija aplikacijoje yra daiktams, todėl nėra matomų adresų bei jų negalima pridėti prie daiktų. Todėl reikalinga papildoma tiesioginė komunikacija sužinoti, kur pasiimti daiktus. Nesant užtrikrinimui, kur daiktas padėtas gali prisidėti papildomas kelias iki paėmimo. Sprendimas veikia tik mobiliąja aplikacija. Žemiau pateikiamas vaizdas iš sistemos daiktų sąrašo grupėje (žr. **pav. 1**)



**pav. 1** „Stuffly“ galimų skolintis daiktų sąrašas

Antras apžvelgtas sprendimas „Peerby“ veikia Olandijoje, Belgijoje, planuoja plėstis į Ameriką. Puslapis veikia tik naršyklėje – programėlė neegzistuoja. Ši platforma pranašesnė nei „Stuffly“, kadangi yra visi skelbimai yra vieši ir naudotojas gali lengviau rasti reikiamą daiktą. Taip pat yra prašymų sistema, į kurią gali atsiliepti naudotojai, turintys išnuomoti reikalingą daiktą. Kitaip nei „Stuffly“ – žemėlapis matomas, ir yra funkcionalumas atidaryti „Google“ žemėlapi. Dalinimasis paremtas „čia ir dabar“ strategija. Pasiskolinimas vyksta žinutės principu savininkui. Sistema palaiko nuoma už mokestį pasirinktoms dienoms, tačiau tai yra jau apmokestinamas funkcionalumas. Žemiau matomas pagrindinis langas, kuriame matome ir nemokamą stalo skolinimąsi, ir nuomą. Pasirinkta kategorija – sportas ir laisvalaikis (žr. **pav. 2**)



pav. 2 „Peerby“ pagrindinis puslapis

lentelė 1 Sprendimų palyginimas

| Lyginimo kriterijai | Peerby              | Stuffly                   | Sukurtas sprendimas             |
|---------------------|---------------------|---------------------------|---------------------------------|
| Platforma           | Tik Naršyklėje      | iOS/Android               | Ir Naršyklėje ir Android        |
| Kalba               | Olandų, Anglų       | Anglų                     | Anglų, Lietuvių                 |
| Rezervavimas        | Nėra                | Rezervavimas kalendoriuje | Rezervavimas kalendoriuje       |
| Kategorijos         | Platus pasirinkimas | Platus pasirinkimas       | Fokusuota į laisvalaikio prekes |
| Paplitimas          | Olandija, Belgija   | Belgija, Vokietija        | Lietuva                         |
| Daiktas žemėlapyje  | Apytikslė vieta     | Nėra                      | Tikslus adresas                 |

Lyginant esamus sprendimus su kuriama sistema išsiskiria 2 pagrindiniai bruožai: 1) Paprastas vietovės atidarymas žemėlapyje 2) Paplitimas – atrasti sprendimai neturi didelės naudotojų bazės Lietuvoje; Tai atsiliepia vertimuose bei galimose kalbose. Sistema palaiko pagrindines funkcijas kaip ir analizuojami konkurentai – kategorijos, žemėlapiai, tiesioginiai pokalbiai su nuomotojais. Kuriamas sprendimas taip pat lengvai pritaikomas ir kitoms kalboms – tik reikalingi vertimai sistemai. Kuriamas sprendimas taip pat paremtas „React Native“, todėl pritaikomas ne tik programėlėms, bet ir naršyklei.

### 1.1.5. Prototipai ir pagalbinių informacija

Kuriant sistemą nebuvo kuriama pagal prototipus ar esamas sistemas. Pirminiai puslapiai buvo perdaromi, remianti asmenine, subjektyvia nuomone dėl naudotojui draugiškos sąsajos.

## 1.2. Techninių galimybių analizė

Sistemos serverio dalis buvo realizuota C# kalba pasinaudojant „ASP.NET Core“ karkasu. Bendravimui su duomenų baze panaudota „EF Core“ ORM biblioteka. Duomenų bazei projekto metu pasirinkta naudoti „SQLite“ tipo reliacinę duomenų bazę dėl paprastumo ir projekto dydžio.

Paveikslėliai saugomi tiesiogiai serverio direktorijoje. Šis saugojimas diegimo metu pakeistas AWS reliacine duomenų baze RDS bei AWS S3 nuotraukų saugojimui.

Naudotojo sąsajos daliai pasirinkta naudoti „TypeScript“ programavimo kalbą su „React Native“ karkasu aplikacijų ir naršyklės suderinamumui. Projekto kūrimo procese buvo naudojama „Expo“, kuris yra paremtas „React Native“ paprastesniam kūrimo ir testavimo procesui naudojant naršyklę ar, naudojant „Expo GO“ smėlio dėžę, patikrinti veikimą su Android įrenginiu. Vėliau pereita prie kūrėjo aplinkacijos išnaudojant EAS Build (2) .

## **2. Projektas**

### **2.1. Reikalavimų specifikacija**

#### **2.1.1. Sistemos funkcijos**

Platformos funkciniai reikalavimai pateikiami UML panaudojimo atvejų diagrama (žr. **pav. 3**)





PA diagramoje matoje, jog yra 3 naudotojo pusės aktoriai Svečias, Registruotas naudotojas ir Administratorius. Išoriniai aktoriai taip par skirtomi į 3 grupes:

1. Autorizavimas
  - a. Firebase – standartinis prisijungimas
  - b. Google, Facebook, Microsoft – išoriniai autorizavimo servais
2. Talpykla – AWS S3 – kreipiamasi papildomai ne tik į duomenų bazę
3. Žemėlapių – GeoCoding

Peržvelgiant svarbiausius panaudojimo atvejus išsiskiria šie:

PA „Redaguoti įrangą“ pateikiamas lentelė 2. Tai yra vienas svarbesnių PA, kadangi susijungia daugiau sistemos įrankių – vietovės pridėjimas, nuotraukų pridėjimas/šalinimas.

lentelė 2 PA „Redaguoti įrangą“ specifikacija

|                                   |                                                               |
|-----------------------------------|---------------------------------------------------------------|
| <b>PA</b>                         | Redaguoti įrangą                                              |
| <b>PA tikslas</b>                 | Pakeisti įrangos įrašo reikšmės                               |
| <b>PA aprašymas</b>               | Įrangos savininkas redaguoja norimus laukus įrangos kortelėje |
| <b>Prieš sąlyga</b>               | Įrangos savininkas                                            |
| <b>Aktoriai</b>                   | Registruotas naudotojas                                       |
| <b>Susiję panaudojimo atvejai</b> | Pridėti nuotraukas, Šalinti nuotraukas, Sukurti vietovę       |
| <b>Po sąlyga</b>                  | Įrangos įrašas redaguotas                                     |

PA „Keisti rezervacijos statusą“ pateikiamas lentelė 3. Serverio dalyje šis veiksmas nėra sudėtingas, tačiau yra daug taisyklių – koki statuso pakeitimą gali naudotojo sąsaja atiduoti.

lentelė 3 PA „Keisti rezervacijos statusą“ specifikacija

|                                   |                                                          |
|-----------------------------------|----------------------------------------------------------|
| <b>PA</b>                         | Keisti rezervacijos statusą                              |
| <b>PA tikslas</b>                 | Pakeisti rezervacijos įrašo statuso reikšmę              |
| <b>PA aprašymas</b>               | Pagal atitinkamą esamą statusą pakeisti į sekantį galimą |
| <b>Prieš sąlyga</b>               | Įrangos savininkas arba rezervacijos kūrėjas             |
| <b>Aktoriai</b>                   | Registruotas naudotojas                                  |
| <b>Susiję panaudojimo atvejai</b> |                                                          |
| <b>Po sąlyga</b>                  | Rezervacijos statusas pakeistas                          |

PA „Sukurti vietovę“ pateikiamas lentelė 4. Svarbus, kadangi yra naudojami išoriniai aktoriai, vartotojas turi būti tinkamai įspėtas, jeigu suvestas adresas netinkamas pagal jokią paiešką

lentelė 4 PA „Sukurti vietovę“ specifikacija

|                     |                                                                                          |
|---------------------|------------------------------------------------------------------------------------------|
| <b>PA</b>           | Sukurti vietovę                                                                          |
| <b>PA tikslas</b>   | Sukurti naują vietovės įrašą                                                             |
| <b>PA aprašymas</b> | Parinkus pavadinimą vietovei, surandama pagal adresą vieta žemėlapyje ir užregistruojama |
| <b>Prieš sąlyga</b> | Įrangos savininkas                                                                       |
| <b>Aktoriai</b>     | Registruotas naudotojas, GeoCoding                                                       |

|                                   |                       |
|-----------------------------------|-----------------------|
| <b>Susiję panaudojimo atvejai</b> |                       |
| <b>Po sąlyga</b>                  | Sukurta nauja vietovė |

PA „Rašyti žinutę“ pateikiamas lentelė 5. Pagrindė šio panaudojimo atvejo pagalba yra realizuojama visa „gyva“ komunikacija tarp vartotojų be jokių reikalavimų perkrovinėti puslapio.

lentelė 5 PA „Rašyti žinutę“ specifikacija

|                                   |                                                                                                                                    |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>PA</b>                         | Rašyti žinutę                                                                                                                      |
| <b>PA tikslas</b>                 | Išsiųsti žinutę kitam naudotojui realiu laiku                                                                                      |
| <b>PA aprašymas</b>               | Išsiunčiama tekstinė žinutė kitam naudotojui, kuri gyvu laiku yra gaunama ir atvaizduojama kaip neperskaityta navigaciniame meniu. |
| <b>Prieš sąlyga</b>               | Egzistuojantis pokalbis tarp naudotojų                                                                                             |
| <b>Aktoriai</b>                   | Registruotas naudotojas                                                                                                            |
| <b>Susiję panaudojimo atvejai</b> |                                                                                                                                    |
| <b>Po sąlyga</b>                  | Išsiųsta žinutė, o gavėjas įspėtas apie naują žinutę                                                                               |

### 2.1.2. Realizacijai keliama reikalavimai

Projekto projektavimo ir realizavimo metu buvo išsikelti šie nefunkciniai reikalavimai:

1. Sistema veikia Anglų ir Lietuvių kalba
2. Sistema prieinama 2 temomis – šviesia ir tamsia
3. Serverinės dalies metodai pasiekiami tik iš autentifikuotų vartotojų
4. Sistema gali būti diegiama arba debesijoje, arba nuosavame serveryje

### 2.1.3. Techninė specifikacija

Kadangi platforma gali būti diegiama tiek debesijos paslaugose, tiek asmeniniame serveryje skiriasi reikalavimai.

Norint pasileisti serverį debesijoje reikalingos šios AWS paslaugos:

- AWS RDS – duomenų bazė, skirta saugoti informacijai
- AWS S3 – nuotraukų/failų saugojimo sprendimas
- AWS ElasticBeanstalk – tinkama mažiausiais parametrais. Reikalingas visai serverio logikai saugoti
- AWS Cloudfront – paverčia HTTP į HTTPS nuorodą. Reikalinga norint naudoti OAuth prisijungimus.

Jeigu norima diegti vietiniame serveryje – sistema daug supaprastėja. Serveriui būtų rekomenduojama turėti: Intel i5, 16GB RAM, 1TB SSD, plačiajuostį internetą suvaldyti srautui.

Naudotojo sąsajos įranga – bent Android 7.0 versija mobiliuosiuose įrenginiuose. Naršyklėse Edge, Firefox, Chrome reikalinga bent 119 šių naršyklių versija.

## 2.2. Projektavimo metodai

### 2.2.1. Projektavimo valdymas ir eiga

Projektas buvo vykdomas iteraciniu pobūdžiu. Išoriniai įrankiai tam nebuvo naudojami. Projektas buvo projektuojamas bei realizuojamas po vieną sistemos dalį pagal bendrą susitarimą su dėstytoju.

### 2.2.2. Projektavimo technologija

Sistema buvo projektuojama naudojantis UML 2.5 standartą. Programinė įranga, kuria buvo projektuojama – „Magic Systems of Systems Architect“ bei „Enterprise Architect“ atgaliniam kodo generavimui (C# į UML)

### 2.2.3. Programavimo kalbos, derinimo, automatizavimo priemonės, operacinė sistema

Naudojamos technologijos kuriamai sistemai:

1. Serverinė dalis:
  - 1.1. Programavimo kalba C#;
  - 1.2. „ASP.NET“ karkasas;
  - 1.3. „Entity framework“ biblioteka;
  - 1.4. „SignalR“ realaus laiko pranešimų biblioteka;
  - 1.5. „Firebase“ biblioteka vartotojų autentifikavimui;
  - 1.6. „SQLite“ operatyviosios atminties duomenų bazė;
  - 1.7. „xUnit“ ir „Moq“ programinio kodo testavimui.
2. Naudotojo sąsaja
  - 2.1. Programavimo kalba „TypeScript“;
  - 2.2. „React Native“ biblioteka;
  - 2.3. „Expo“ karkasas;
  - 2.4. „Axios“ biblioteka HTTP užklausoms
  - 2.5. „SignalR“ realaus laiko pranešimų biblioteka.
3. Debesijos paslaugų tiekėjas „Amazon Web Services“
4. Kodo saugykla „Github“

lentelė 6 Programinė įranga

| Kriterijus                                     | Programinė įranga                                |
|------------------------------------------------|--------------------------------------------------|
| Operacinė sistema                              | Windows 11                                       |
| Specifikacija                                  | Intel I5-1135G7 2,4GHz; 24GB Darbinės atminties; |
| Serverinės ir naudotojo sąsajos kūrimo įrankis | Visual Studio Code su pagalbinais įskiepiais     |
| Serverinės dalies testavimo įrankis            | Visual Studio 2022                               |
| Duomenų bazės valdymo įrankis                  | „SQL Server Management Studio“                   |

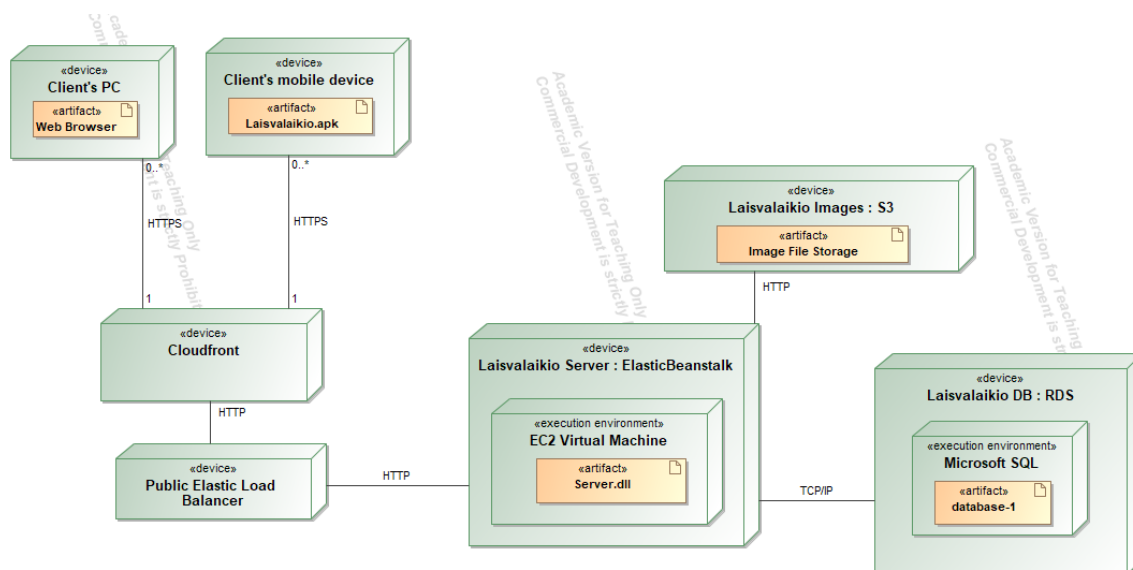
## 2.3. Sistemos projektas

### 2.3.1. Statinis sistemos vaizdas

Paveikslėlyje **pav. 4** matoma diegimo diagrama, nurodanti fizinį išdėstymą. Serveris bei duomenų talpyklos yra AWS debesijos paslaugose.

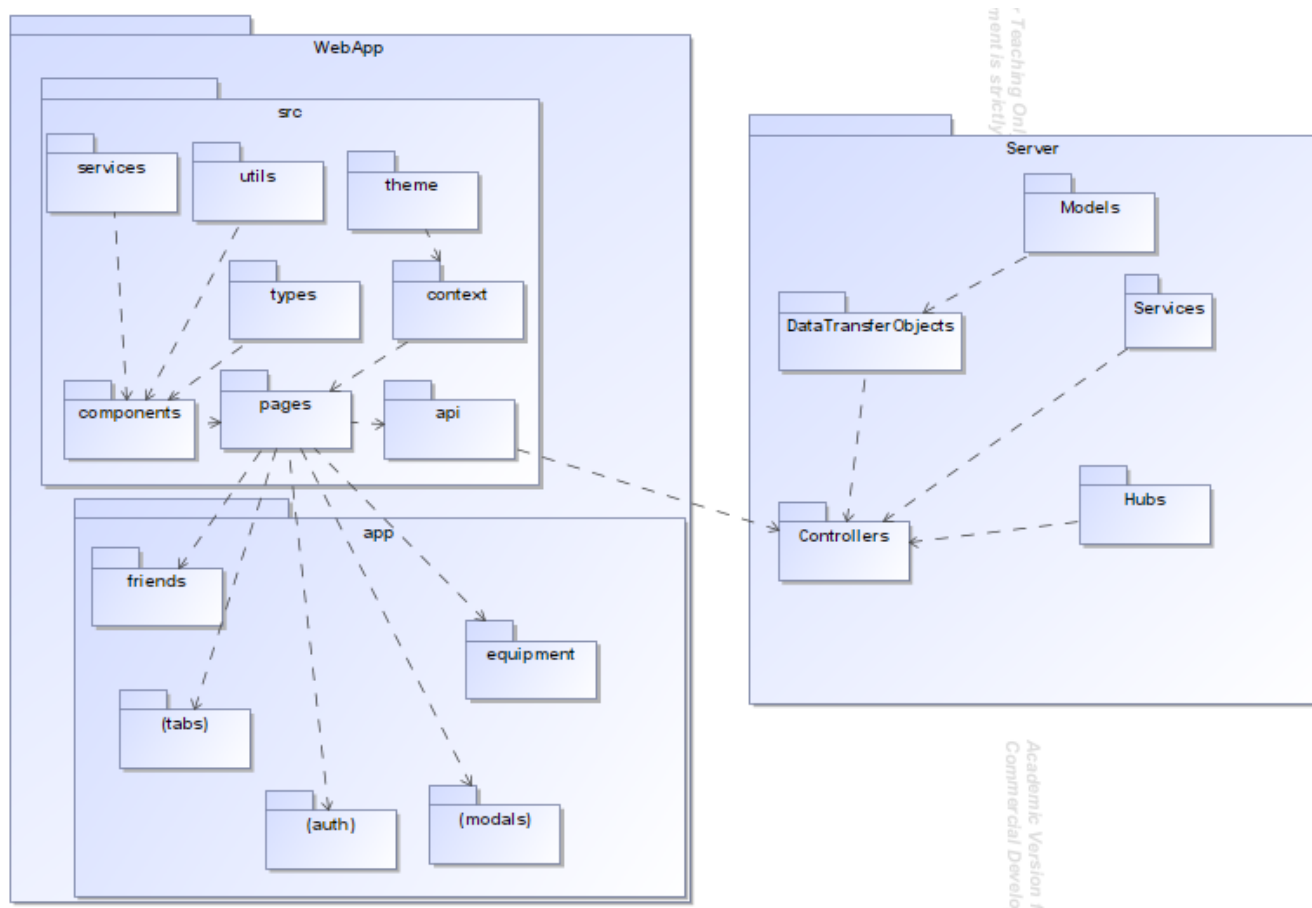
Sistemos serverinė dalis yra patalpinta AWS debesijos tiekėjo serveriuose. Naudotojo sąsaja pasiekama per localhost naršyklę arba Android aplikacija. Nurodytas diegimo planas yra išskirstytas į ElasticBeanstalk – serverio logikos talpyklą; RDS duomenų bazę; S3 – nuotraukų saugojimui ir Cloudfront – turinio pristatymo tinklas, atsakingas užtrikrinti saugų ryšį su išoriniais servais bei naudotojo sąsaja. Serverinė dalis turi alternatyvą diegimui – lokalūs serveriai. Tuomet dingsta

reikiamybė išskaidytiems resursams ir užtenka tik sudiegti serverio logikos pusę. Programinis kodas prisitaiko sudėlioti aplankus bei SQLite duomenų bazę atmintyje. Vienintelis reikalavimas išlieka – tikras sertifikatas pasirašymui, jog būtų užtikrintas HTTPS protokolo ryšys su išoriniais servisais.



pav. 4 Diegimo diagrama

Paveikslėlyje pav. 5 pateikiama sistemos paketų diagrama. Naudojama įprasta MVC struktūra, kai modelių ir valdiklių dalis egzistuoja serverinėje pusėje, o visi vaizdai/puslapiai yra apdorojami naudotojo sąsajos pusėje. Serverinėje dalyje taip pat naudojami duomenų perkėlimo objektai, apribojantys grąžinamą informaciją atgal į naudotojo sąsają (pagrindė negrąžinami ID įvairioms operacijoms, kadangi pagal tai būtų gali įvykdyti atakas duomenų bazei. Naudotojo sąsaja taip pat seka Expo diktuojamą struktūrą. Puslapiai sistemoje yra dėliojami pagal „app“ aplankalo failus/aplankalus. Šiuose objektuose yra saugoma teisių priskyrimai, navigaciniai keliai bei kvietimai į puslapius. Patiems puslapiams buvo pasirinkta moduli struktūra išnaudojant komponentus dėl didesnio pakeičiamumo.



pav. 5 Paketų diagrama

Toliau pateikta duomenų bazės schema (žr. **pav. 6**), nurodanti modelių struktūrą bei esamus ryšius tarp jų.

Visa sistema sukasi aplink 2 pagrindines lenteles: „AspNetUsers“ ir „Equipment“. Kas sistemoje ir yra svarbiausi komponentai.

Kitos lentelės pagrinde tik padeda/papildo aukščiau paminėtas lenteles.

„Equipment“ lentelę papildo – kategorijos. Kategorijos iškeltos į atskirą lentelę dėl patogesnio papildymo iš administratoriaus pusės bei dinaminio paveikslėlių pritaikymo.

Įrangos nuotraukos – visiškai pagalbinė lentelė, skirta saugoti ne vieną URL, bet daugelį skirtingų.

Atsiliepimai – koreliuoja tik su įranga. Daugiau sistemoje atskirai patikrinti jų negalima.

Vietovės – jos yra priskiriamos naudotojui, bet vienintelis panaudojimas sistemoje yra priskirti prie įrangos, atvaizdavimui žemėlapyje.

Situacija panaši su naudotojo lentele. Visos kitos lentelės prisiraša prie šios pagrindinės lentelės. Kaip pvz pokalbiai, žinutės, perskaitymai ir pagrindinis rezervavimai.



### 2.3.2. Dinaminis sistemos vaizdas

Sistemos dinaminis vaizdas pasirinktas vaizduoti sekų diagramomis, nurodyti visą eigą tarp komponentų ir skirtingų iškvietimų. Vienas atvejis yra atvaizduojamas būsenų diagrama, kadangi vienas panaudojimo atvejis/sekų diagrama taptų chaotiška dėl daugelio taisyklių. Pasirinkti keli pagrindiniai sistemos momentai atvaizduoti diagramomis, kadangi kitos operacijos yra atominės be papildomų išsiplėtimų.

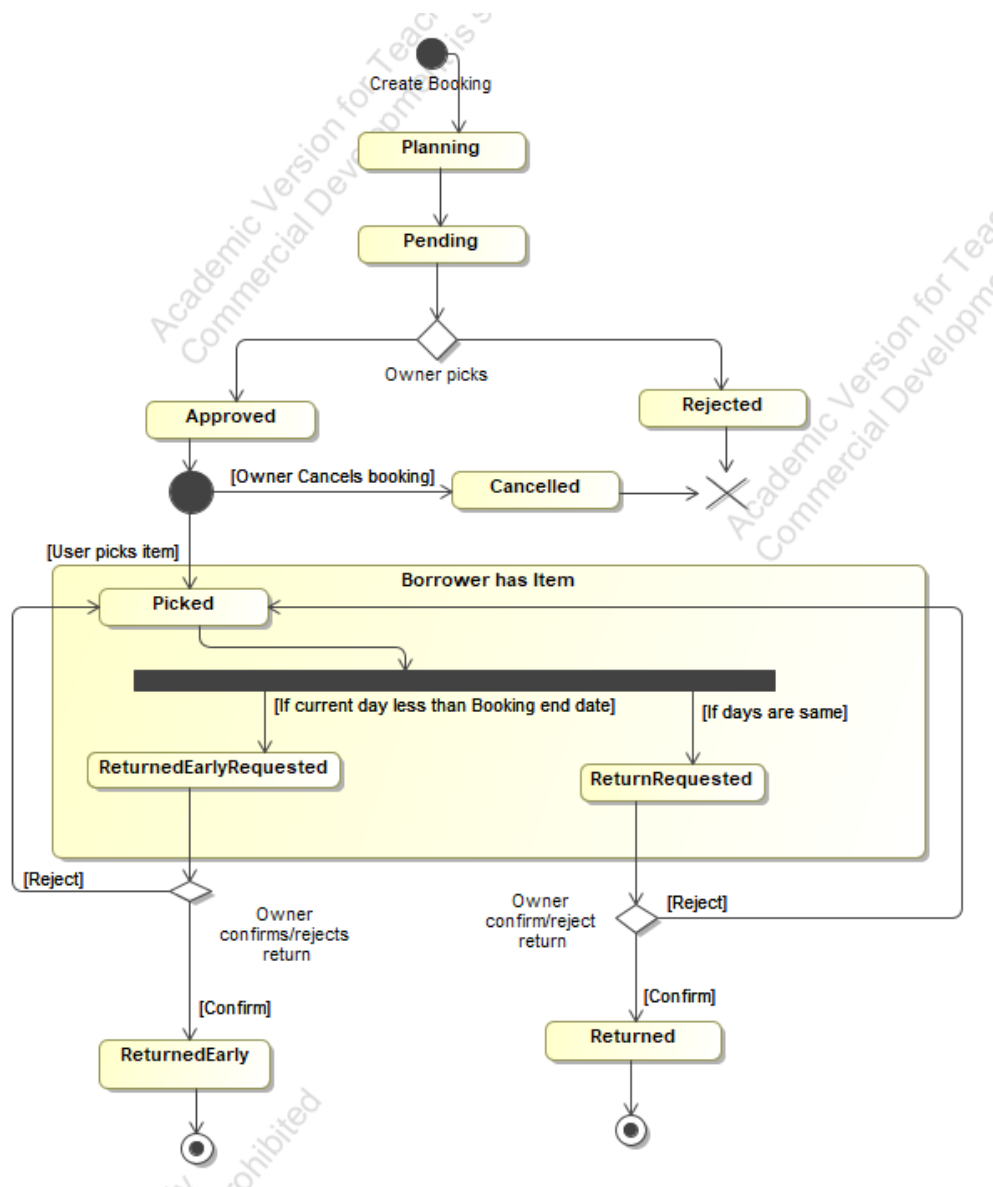
PA „Keisti rezervacijos statusą“ pasirinkta atvaizduoti būsenų diagrama (3), kadangi priklausomai nuo kokio tipo aktorius atlieka veiksmus – galimybės skyla į daugelį šakų. Diagramoje matome (žr. **pav. 7**), jog sukūrus rezervaciją ji turi iš viso 10 skirtingų būsenų, iš kurių 4 yra galutinės. Planavimo ir laukimo fazėje rezervuotojas atlieka visus veiksmus ir laukia savininko. Savininkas tuomet turi 2 pasirinkimus. Arba atmesti ir įrašas yra ištrinamas, arba patvirtinti. Kadangi rezervacija gali būti patvirtinta iš anksto ilgą laiką – Savininkas turi teisę atšaukti rezervaciją – kuomet taip pat yra ištrinama rezervacija.

Tačiau patvirtinus rezervaciją, naudotojas iškėlęs užklausą, gali pažymėti įrangą kaip paimtą. Šiame žingsnyje savininkas jau nebeturi galimybės keisti statuso. Naudotojas gali arba grąžinti anksčiau arba laiku. Kokia opcija jam galima priklauso nuo rezervacijos pabaigos ir esamos datos. Prie grąžinimo naudotojas turi galimybę pridėti nuotrauką – užtvirtinti, jog įrangą saugiai grįžo į buvusią poziciją.

Savininkas tuomet privalo patvirtinti grąžinimą arba jį atmesti, kuomet daiktas grįžta į „Paimtas“ statusą ir naudotojas gali bandyti iš naujo teikti užklausą atidavimui, galbūt dabar su nuotraukos užtvirtinimu.

Grąžinti rezervavimai yra išimami iš prieinamumo tikrinimo ir kiti naudotojai gali rezervuoti tą pačią įrangą.

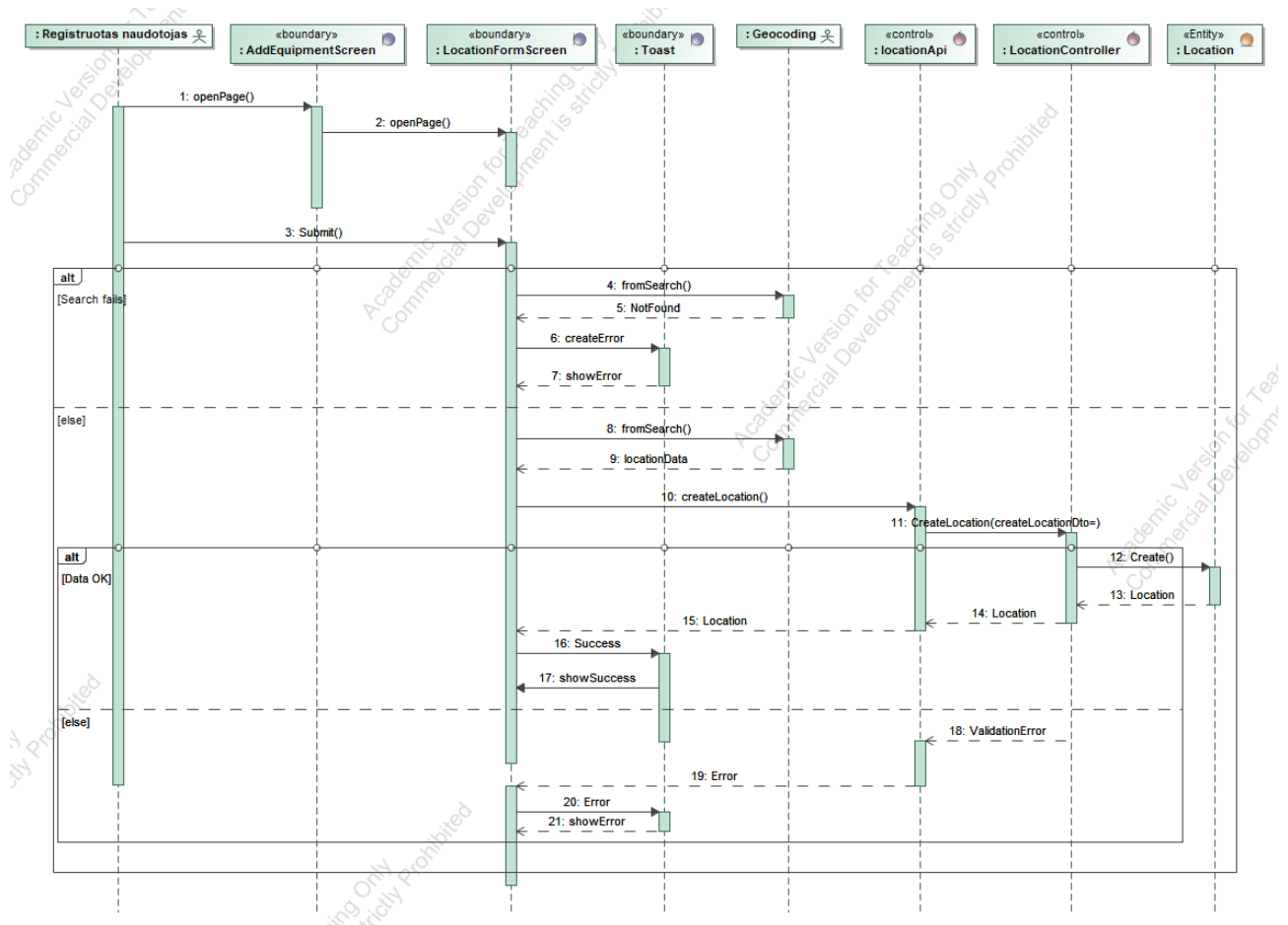




**pav. 7** PA „Keisti rezervacijos statusą“ būsenų diagrama

Sekančios 3 sekų diagramos yra susiję, kadangi vainikuojasi paskutinėje diagramoje – įrangos sukūrimu.

Pirmoji yra Vietovės sukūrimo diagrama (žr. **pav. 8**). Iš Įrangos kūrimo lango yra mygtukas papildomam langui ant viršaus – pridėti naują vietojė. Suvedama pavadinimas, aprašymas ir tuomet paieškos laukelyje yra ieškomas norimas adresas. Komponentas yra tikrinamas dėl klaidų ir šios yra atvaizduojamos atgal naudotojui per Toast biblioteką. Bet jeigu adresas yra randamas – surenkami reikalingi Geo lokacijos duomenys ir per API iš naudotojo sąsajos įrašas keliauja į serverinę dalį. Ten įvyksta dar vienas patikrinimas įterpimo metu. Jeigu nesėkmingas – naudotojui taip pat grįžta klaidos pranešimas. Teisingų duomenų atveju grąžinama nauja vietovė, ją gali pasirinkti iš sąrašo ir vartotojas taip pat gauna pranešimą, jog viskas teisingai išsisaugojo.

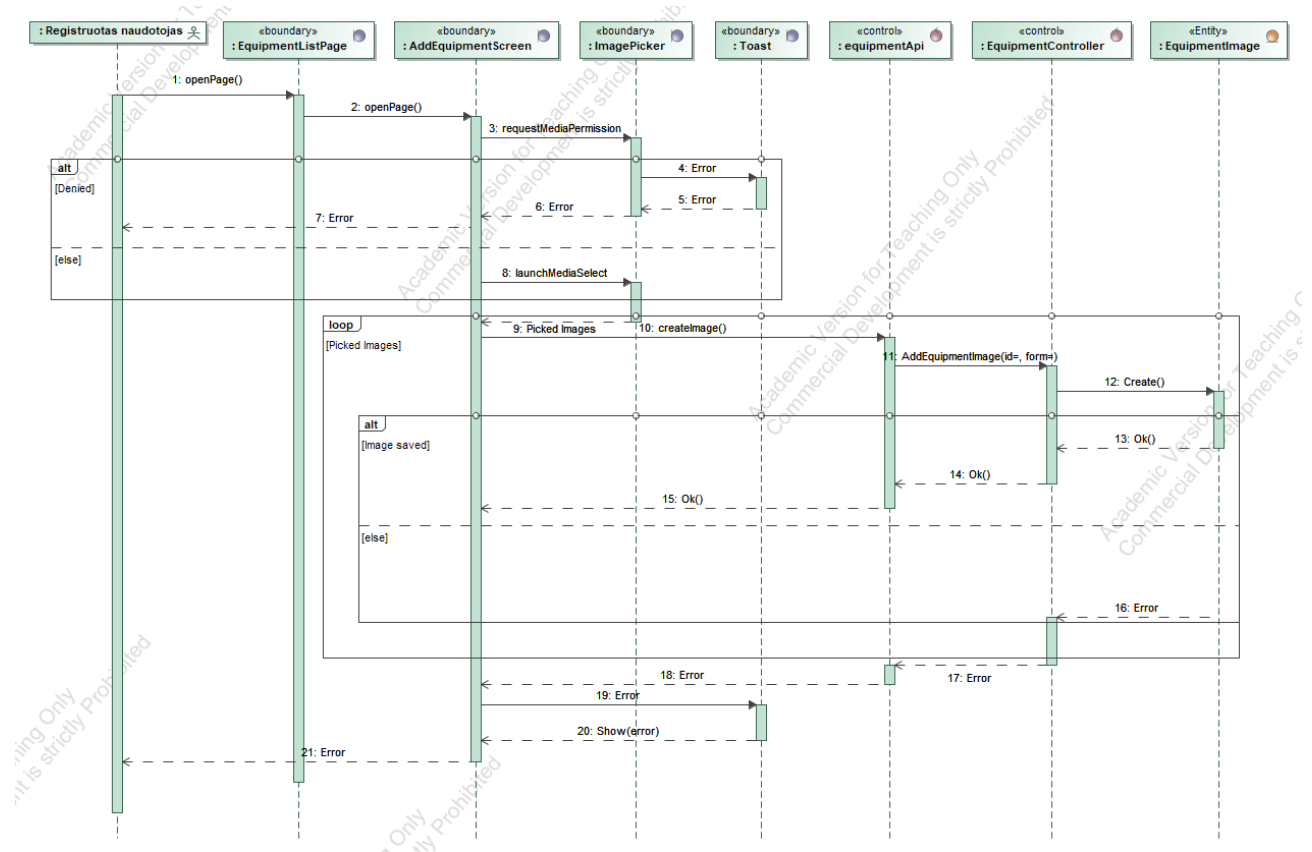


pav. 8 PA „Sukurti vietovę“ sekų diagrama

Sekanti sekų diagrama, kuri bus rekomenduojama (nebūtina) įrangos sukūrimui – Pridėti nuotraukas (žr. pav. 9).

Viskas vyksta iš įrangos pridėjimo lango. Pirmas tikrinimas bandant pridėti nuotraukas – mūsų klausia dėl leidimo naršyti po kompiuterį ar mobiliųjį telefoną. Nesutikus – pridėti nuotraukų negalima.

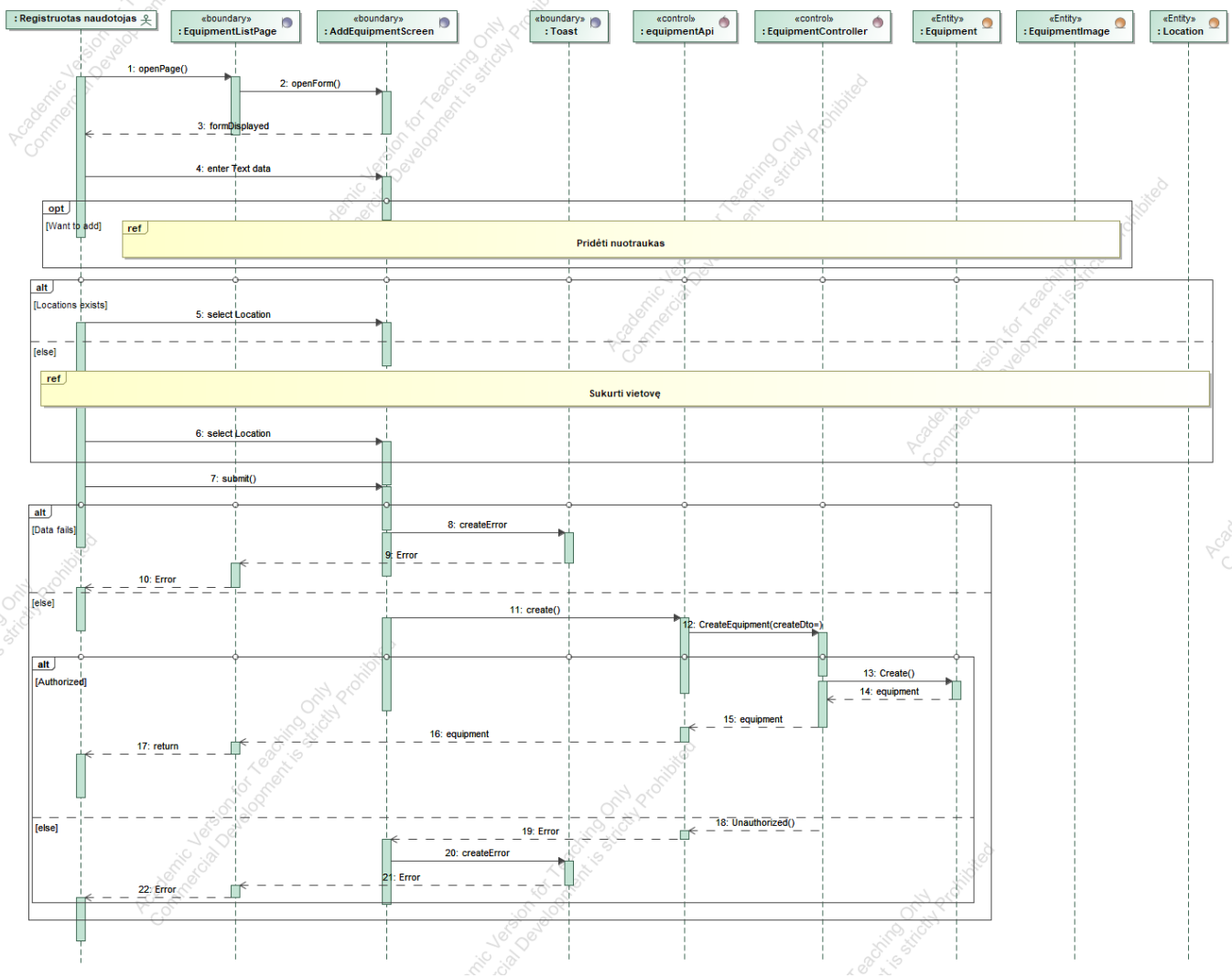
Sutikus atidaroma turima galerija ar paskutinis aplankalas su filtru nuotraukų plėtiniam. Teisingai susikūrus nuotraukai (galimi jos redagavimai) ši nuotrauka pridedama į serverio pusę ir išsaugoma. Visos nuotraukos yra saugomos atskirai, todėl 1 blogai nuotraukai įsimaišius (kurios negalėjo apdoroti) visos kitos bus suimportuotos. Papildomų pranešimų dėl suimportuotų nuotraukų nėra. Jos atsiranda kuriamos įrangos lange.



**pav. 9** PA „Pridėti nuotraukas“ sekų diagrama

Pagrindinė sekų diagrama – įrangos sukūrimas (žr. **pav. 10**) tampa paprasta, kadangi papildomi žingsniai iškelti į atskiras sekų diagramas. Supildžius tekstinius duomenis, pasirinkus kategoriją, vietovę, pridėjus nuotraukas teikiame registravimui. Naudotojo sąsajoje yra patikrinami privalomieji laukai, esant problemoms – metama klaida per „Toast“.

Problemų neaptikus yra siunčiama į serverį kur pirmiausia sukuriamą įrangą, tuomet registruojamos nuotraukos. Pavykus – įranga užregistruojama ir vartotojas yra grąžinamas į Įrangų sąrašo langą. Esant klaidoms – gaunamas „Toast“ klaidos pranešimas ir liekame tame pačiame lange.

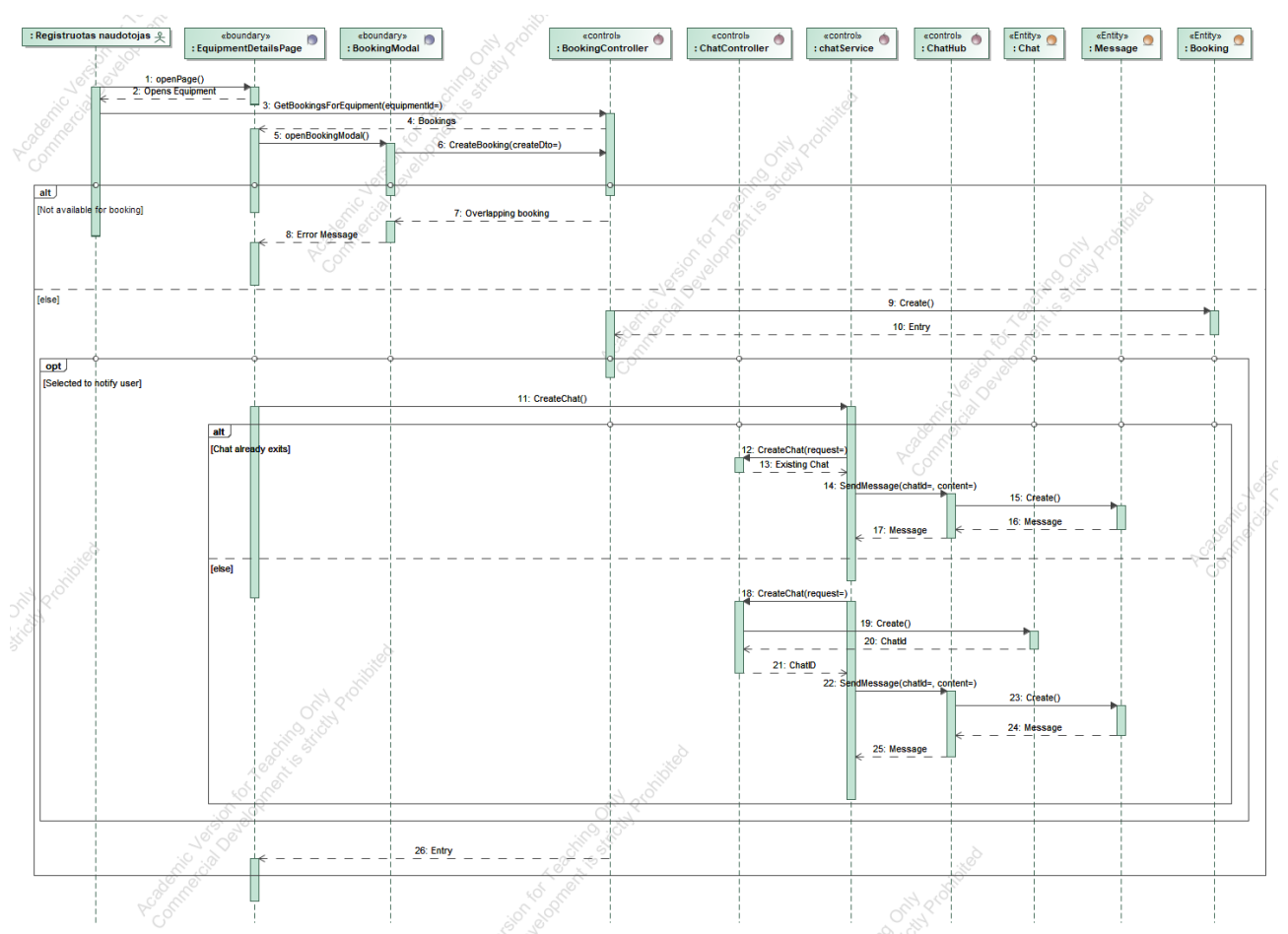


pav. 10 PA „Pridėti įrangą“ sekų diagrama

PA „Sukurti Rezervaciją“ turi gana didelį papildomą funkcionalumą, kuris įvykdomas tuo pačiu mygtuku. (žr. pav. 11). Atsidarius įrangos kortelę žemiau matome rezervacijų kalendorių, jame mygtukas „Pridėti rezervaciją“. Tai atidaro modalų langą. Informacijos nedaug – pradžios, pabaigos datos bei pastabos. Tačiau yra dar „Ar išpėti savininką žinute“. Pasirinkus varnelę ir bandant registruoti rezervaciją informacija yra siunčiama į serverį. Čia yra patikrinama ar rezervacijos dienos nesikerta su jokia kita egzistuojančia rezervacija. Jeigu taip – grąžinamas klaidos klientui, jog datos kertasi.

Priešingu atveju – įrašas yra užregistruojamas ir grąžinamas per valdiklį. Procesui vykstant toliau yra tikrinama ar egzistuoja pokalbio įrašas tarp rezervuotojo ir savininko. Jeigu nėra sukuriamas, kitaip – grąžinamas ID. Iš sistemoje numatyto šablono yra sugeneruojama žinutė savininkui, pridedama nuoroda mini rezervacijos langui ir išsiunčiama.

Vėliau savininkui atsidarius žinutes – bus ir šabloninė žinutė, ir greita nuoroda į būtent tą rezervacijos įrašą.



pav. 11 PA „Sukurti rezervaciją“ sekų diagrama

### **3. Testavimas**

#### **3.1. Testavimo planas**

Testavimo atlikimo tvarka : Statinė kodo analizė bei rankinis testavimas buvo atliekamas viso projekto kūrimo metu, vienetų testai atlikti įsitikinus dėl galutinės kontrolerių versijos.

#### **3.2. Testavimo kriterijai**

Statinė kodo analizės kriterijai:

- Rašomas kodas atitinka esamą struktūrą
- Stengiamasi pakartotinai panaudoti jau esamus komponentus
- Aiškūs kintamųjų pavadinimai
- Naudojami tipizuoti kintamieji duomenų apsikeitimui

Integraciniai testai:

- Testai padengia ne tik sėkmingą operacijos scenarijų
- Testai padengta serverio pusė turi siekti bent 70% kontrolerių/servisų metodų.
- Visi testai įvykdomi sėkmingai

Rankinis sąsajos testavimas:

- Patikrinama įprastas sistemos naudojimo atvejis
- Panaudojamos visos prieinamos funkcijos

#### **3.3. Integracinis testavimas**

Integracinius testus buvo pasirinkta rašyti „Controllers“ sistemos daliai, kadangi visa komunikacija su duomenų baze vyksta šioje dalyje. Duomenų bazės veiksmai padengiami iš „EF Core“ pusės.

Buvo naudojama „XUnit“ bei „Moq“ (4) bibliotekos testams atlikti. Kiekvienam testui buvo kuriami tam testui reikalingi duomenys pasinaudojant „Moq“ biblioteka.

Prie „Controllers“ taip pat buvo parašyti testai „TokenService“ klasei, kuri atsakinga už JWT žetonų generavimą. Bendras sistemos padengimas pavaizduotas paveikslėlyje žemiau (žr. **pav. 12**):

|                            |         |             |
|----------------------------|---------|-------------|
| Server                     | 25.61%  | <div></div> |
| Controllers                | 72.60%  | <div></div> |
| BookingController.cs       | 81.68%  | <div></div> |
| CategoryController.cs      | 93.53%  | <div></div> |
| ChatController.cs          | 73.23%  | <div></div> |
| EquipmentController.cs     | 82.50%  | <div></div> |
| FriendshipController.cs    | 71.43%  | <div></div> |
| LocationController.cs      | 79.72%  | <div></div> |
| LoginController.cs         | 49.91%  | <div></div> |
| MobileOAuthController.cs   | 70.63%  | <div></div> |
| ReviewController.cs        | 93.15%  | <div></div> |
| StorageController.cs       | 71.92%  | <div></div> |
| UserController.cs          | 74.92%  | <div></div> |
| > Data                     | 100.00% | <div></div> |
| > DataTransferObjects      | 98.98%  | <div></div> |
| > Hubs                     | 90.16%  | <div></div> |
| > MappingProfiles          | 100.00% | <div></div> |
| > Migrations               | 0.00%   | <div></div> |
| > Models                   | 93.70%  | <div></div> |
| Program.cs                 | 0.00%   | <div></div> |
| Services                   | 74.01%  | <div></div> |
| FirebaseAuthClient.cs      | 0.00%   | <div></div> |
| FirebaseAuthService.cs     | 48.86%  | <div></div> |
| FirebaseTokenVerifier.cs   | 0.00%   | <div></div> |
| IGoogleIdTokenValidator.cs | 18.75%  | <div></div> |
| > Storage                  | 96.24%  | <div></div> |
| TokenService.cs            | 100.00% | <div></div> |

**pav. 12** Serverio dalies testų padengimas

Testais buvo padengta 72% valdiklių.

Iš viso buvo parašyta 291 testai, iš kurių 291 sėkmingi. Dalį testų vykdomo eigos galite matyti paveikslėlyje žemiau (žr. **pav. 13**)

```

✓ Server.Tests (net9.0) 20.4s
> ✓ Server.Tests 39ms
✓ Server.Tests.Controllers 15.5s
> ✓ BookingControllerTests 1.5s
✓ CategoryControllerTests 404ms
  ✓ GetCategories_ReturnsAllCategories 36ms
  ✓ GetCategory_ValidId_ReturnsCategory 150ms
  ✓ GetCategory_InvalidId_ReturnsNotFound 2.0ms
  ✓ CreateCategory_ValidData_ReturnsCreatedCategory 13ms
  ✓ CreateCategory_UsesNormalizedNameAsSlug 3.0ms
  ✓ CreateCategory_InvalidNormalizedSlug_ReturnsBadRequest 3.0ms
  ✓ CreateCategory_DuplicateSlug_ReturnsBadRequest 11ms
  ✓ UpdateCategory_ValidData_ReturnsNoContent 3.0ms
  ✓ UpdateCategory_InvalidId_ReturnsNotFound 14ms
  ✓ UpdateCategory_DuplicateSlug_ReturnsBadRequest 30ms
  ✓ DeleteCategory_ValidId_ReturnsNoContent 9.0ms
  ✓ DeleteCategory_InvalidId_ReturnsNotFound 35ms
  ✓ DeleteCategory_WithSubcategories_ReturnsBadRequest 12ms
  ✓ DeleteCategory_WithEquipment_ReturnsBadRequest 9.0ms
  ✓ GetSubcategories_ValidId_ReturnsSubcategories 70ms
  ✓ GetSubcategories_InvalidId_ReturnsEmptyList 4.0ms
> ✓ ChatControllerTests 1.2s
✓ EquipmentControllerTests 2.7s
  ✓ GetEquipment_ReturnsAllEquipment 95ms
  ✓ GetEquipment_ById_ReturnsEquipment 147ms
  ✓ GetEquipment_InvalidId_ReturnsNotFound 1.3s
  ✓ CreateEquipment_ValidData_ReturnsCreatedEquipment 326ms
  ✓ UpdateEquipment_ValidData_ReturnsNoContent 20ms
  ✓ UpdateEquipment_NonOwner_ReturnsForbidden 57ms
  ✓ DeleteEquipment_ValidId_ReturnsNoContent 56ms
  ✓ DeleteEquipment_NonOwner_ReturnsForbidden 13ms
  ✓ GetEquipmentByOwner_ReturnsOwnerEquipment 188ms
  ✓ GetEquipmentByOwner_NoEquipment_ReturnsEmptyList 18ms
  ✓ GetEquipment_NoUserClaim_ReturnsUnauthorized 53ms
  ✓ AddEquipmentImage_NoUserClaim_ReturnsUnauthorized 13ms
  ✓ AddEquipmentImage_NullFile_ReturnsBadRequest 14ms
  ✓ AddEquipmentImage_ValidFile_ReturnsOk 71ms
  ✓ DeleteEquipmentImage_EquipmentNotFound_ReturnsNotFound...
  ✓ DeleteEquipmentImage_NotOwner_ReturnsForbidden 25ms
  ✓ DeleteEquipmentImage_ImageNotFound_ReturnsNoContent 14ms
  ✓ DeleteEquipmentImage_ValidImage_ReturnsNoContent 154ms
  ✓ UpdateEquipment_NotFound_ReturnsNotFound 19ms
  ✓ DeleteEquipment_NotFound_ReturnsNotFound 107ms
> ✓ FriendshipControllerTests 573ms
> ✓ LocationControllerTests 1.9s
> ✓ LoginControllerTests 2.3s

```

pav. 13 Testų vykdymo eiga



## 4. Dokumentacija naudotojui

### 4.1. Apibendrintas sistemos galimybių aprašymas

Platforma yra padalinta į 3 pagrindines dalis. Pirmoji yra paprasčiausia – Svečio sąsaja. Šio statuso naudotojai turi galimybę tik registruotis arba prisijungti, naudojantis 3-ios šalies tiekėją. Taip pat keisti sistemos kalbą bei temą.

Antroji dalis yra plačiausia bei viduje dar skyla į 2 atskiras dalis. Tai yra Registruotas naudotojo sąsaja. Pirmas žingsnis galima prisijungti bei atsijungti. Toliau galimi veiksmai yra: pridėti kitą naudotoją kaip draugą, pradėti pokalbį, peržiūrėti įrangas, registruoti rezervacijas, keisti jų statusą, palikti atsiliepimą. Antroji dalis būtų įrangos savininkas – atsiveria galimybės pridėti įrangą, ją redaguoti, tvirtinti/atmesti rezervacijas, tvirtinti gražinimus.

Abiems tipams registruotų naudotojų yra galimybės – peržiūrėti įrangas žemėlapyje, peržiūrėti rezervacijų kalendorių, keisti savo profilį, nustatymų nuostatas.

Paskutinis naudotojo dalis – administratorius. Jam galioja visos prieš tai buvusių naudotojų rolės bei galimybė pridėti/redaguoti esamas kategorijas – sistemoje tai vienintelis administratoriaus darbas.

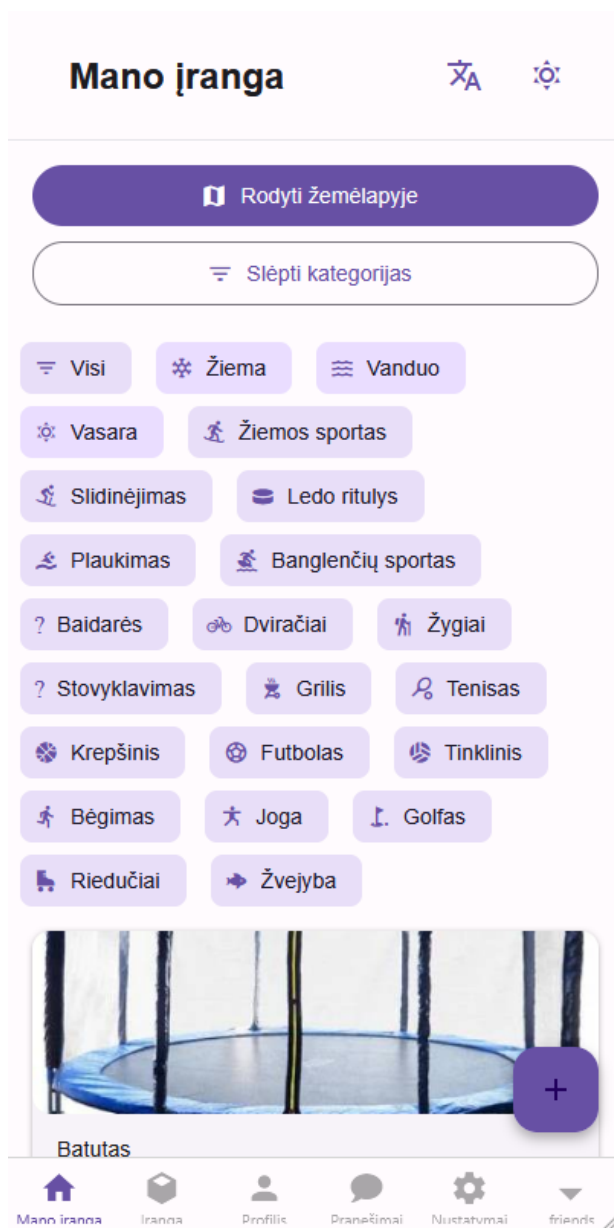
### 4.2. Naudotojo vadovas

Atsidarius programėlę matomas tik prisijungimo langas su galimybe prisijungti per išorines sistemas.

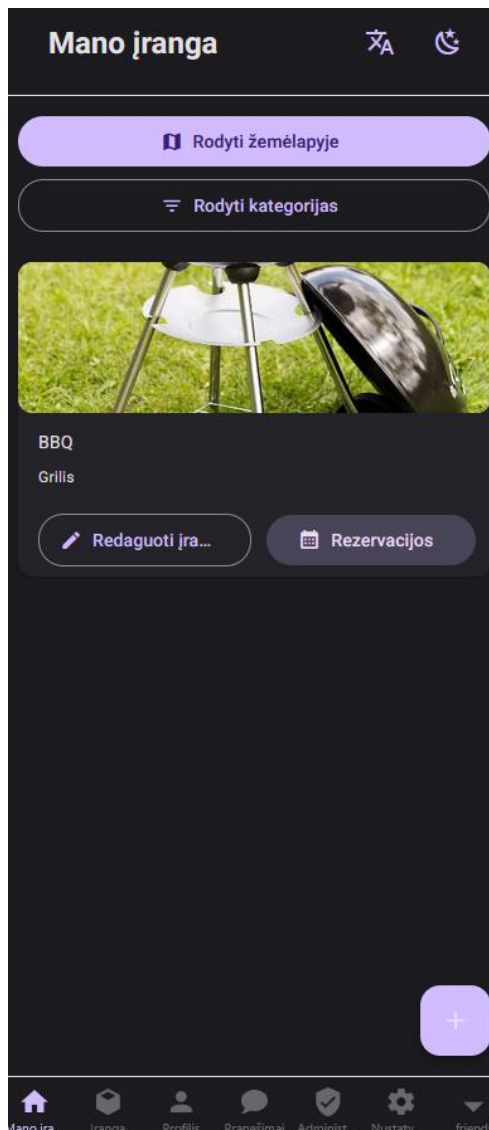
Viršuje kampe matoma nustatymai kalbai bei temai pakeisti. (žr. **pav. 14**)

pav. 14 Prisijungimo langas

Prisijungus prie paskyros, naudotojas nukreipiamas į sistemos pagrindinį langą – mano įrangą. Čia matoma Žemėlapių mygtukas, kategorijų filtrai, kuriuos galima paslėpti dėl matomumo, įrangos sąrašas, mygtukas pridėti naujai įrangai bei apačioje matomas navigacinis meniu tarp skirtingų skirtukų. (žr. pav. 15) Toks pat vaizdas tik tamsia tema matomas kitame paveikslėlyje (žr. pav. 16)



pav. 15 Pagrindinis langas – Mano įranga



**pav. 16** Pagrindinis langas tamsiąja tema

Pasirinkus iš sąrašo ar iš pačios įrangos kortelės – įrangą galime redaguoti. Naudojamas langas yra identiškas naujos įrangos sukūrimui. Matomi keli informaciniai laukai, kuriuos galime suvesti ranka: pavadinimas, aprašas. Toliau vyksta priskyrimai – kategorija iš numatyto sąrašo, nuotraukų užkelimai bei vietovės pasirinkimas arba naujos pridėjimas. (žr. **pav. 17**). Pasirinkus pridėti naują vietovę atsiranda papildomas modalinis langas su vietos pavadinimu, aprašu bei paieškos lauku. Paieškos lauke vedame adresą ir galime pasitikrinti tai žemiau esančiame žemėlapyje (žr. **pav. 18**).



 Edit Equipment

Pavadinimas

BBQ

Aprašymas

BBQ

Kategorija

Grilis

Vieta

Namai

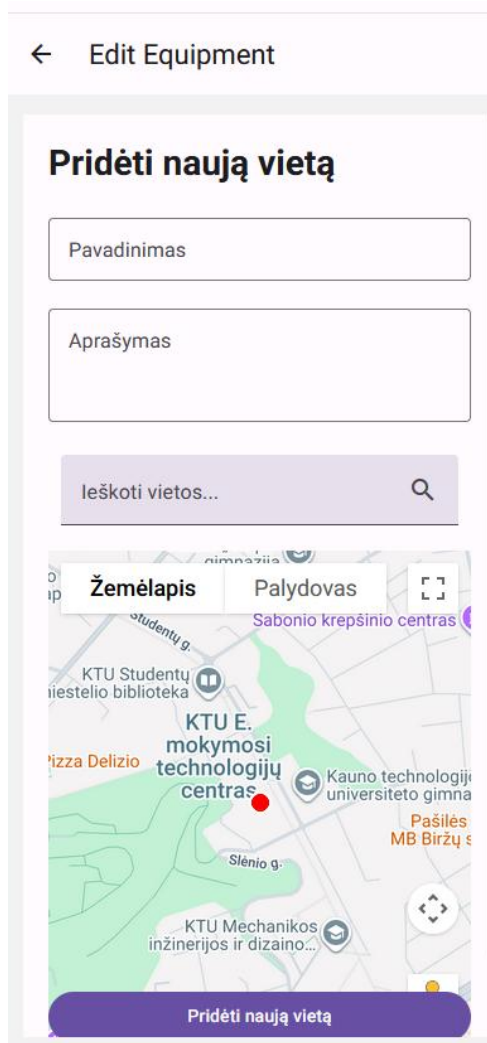
Sukurti naują vietą

Nuotraukos



Atnaujinti įrangą

**pav. 17** Įrangos pridėjimas/redagavimas

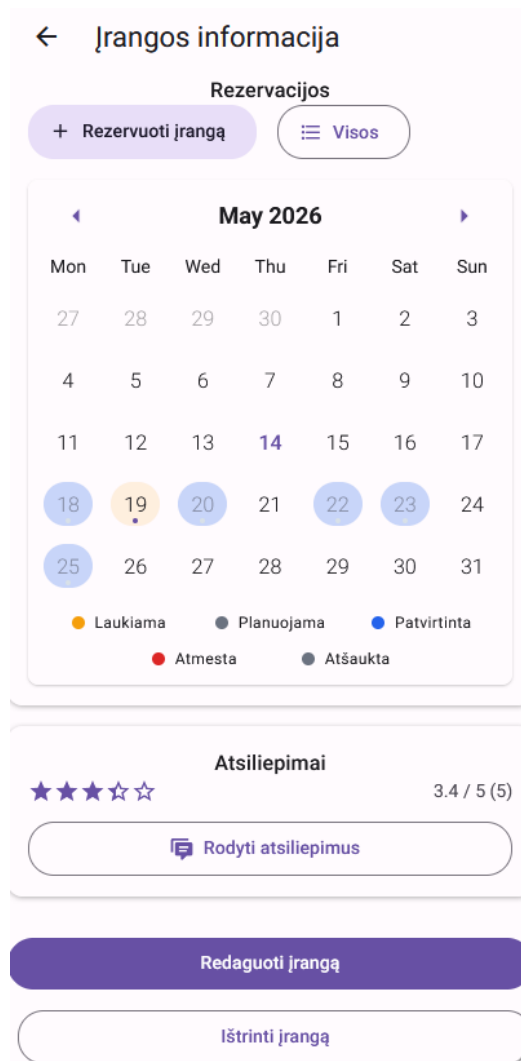


**pav. 18** Naujos vietovės pridėjimas

Redagavus įrangą ar atsidarius ją iš pagrindinio meniu matosi jos detalės (žr. **pav. 19** ir **pav. 20**). Čia atvaizduota bazinė informacija, kuri matoma ir redagavimo lange. Yra nedidelis žemėlapis parodyti įrangos vietą. Prie informacijos taip pat matomas savininkas ir ar įranga dabar yra prieinama. Prieinamumas patikrinimas pagal rezervacijas. Ar jau yra paimtas/turi patvirtintas registracijas. Šios rezervacijos matomos žemiau kortelėje kaip kalendorius su spalvų legenda. Taip pat čia randami ir atsiliepimai. Jeigu naudotojas yra įrangos savininkas – taip pat matomi mygtukai redaguoti/ištrinti įrangą.

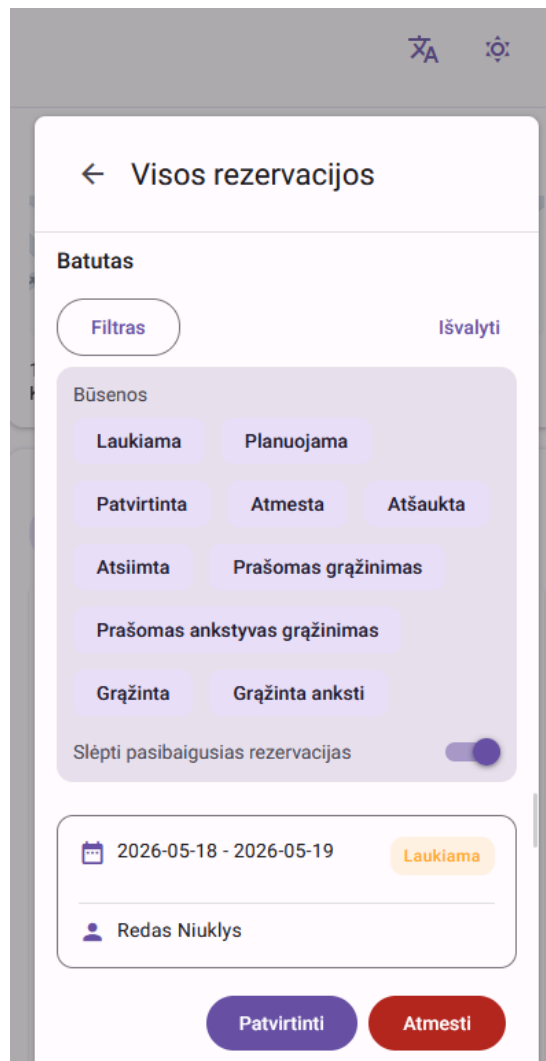


pav. 19 Įrangos informacijos langas 1/2



**pav. 20** Įrangos informacijos langas 2/2 Rezervacijos, Atsiliepimai, Redagavimas

Pasirinkus „Visos“ rezervacijų blokelyje atsidaro sąrašas su visomis esamomis rezervacijomis. Yra galimybė filtruoti pagal esamą statusą. (žr. **pav. 21****pav. 14**) Įrangos savininkas turi papildomus mygtukus prie kiekvienos rezervacijos ją patvirtinti, atmesti. Gražinimo atveju – tai patvirtinti. Filtras, kadangi, užima nemažą dalį ekrano – galimas suskleisti atsifiltravus.



**pav. 21** Rezervacijų sąrašas

Įrangos detalių sąrašė šalia pasirinkus mygtuką „Nauja rezervacija“ atsidaro modalinis langas (žr. **pav. 22**). Suvedamos norimos datos bei pastabų/užrašų blokas. Jeigu naudotojas yra savininkas – automatiškai patvirtinama rezervacija. Kiti naudotojai taip pat matys mygtuką „Informuoti savininką pokalbiu“. Tuomet parašoma žinutė į esamą pokalbį arba sukuriamas naujas pokalbis tarp rezervuojančio ir savininko.



← Nauja rezervacija

**Batutas**

✓ Kaip savininkas, jūsų rezervacija bus automatiškai patvirtinta

**Pradžios data**

2026-05-14

**Pabaigos data**

2026-05-14

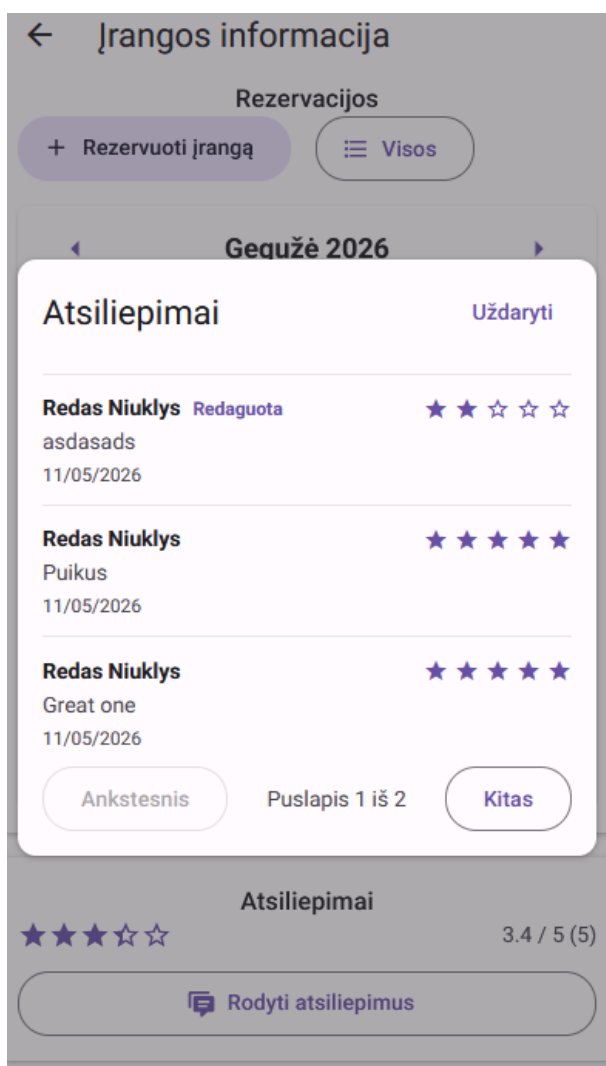
**Pastabos**

Pridėkite papildomos informacijos apie savo rezervaciją...

Rezervuoti

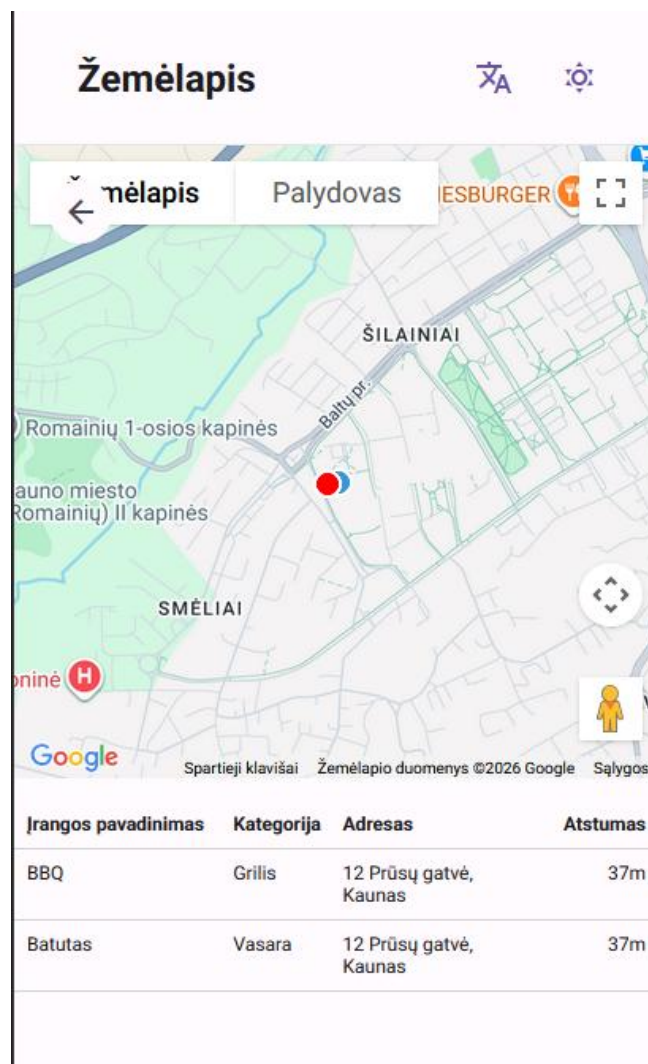
**pav. 22** Nauja rezervacija

Iš įrangos puslapio taip pat galima pasirinkti „Rodyti atsiliepimus“ ir atsidaro modalinis langas (žr. **pav. 23** **pav. 14**). Atsiliepimai yra vizualiai puslapiuojami, neperkrauti naudotojo sąsajos. Jeigu naudotojas turi rezervaciją, kurią atidavė atgal savininkui – atsiranda papildomos eilutės įverčiui bei atsiliepimui.



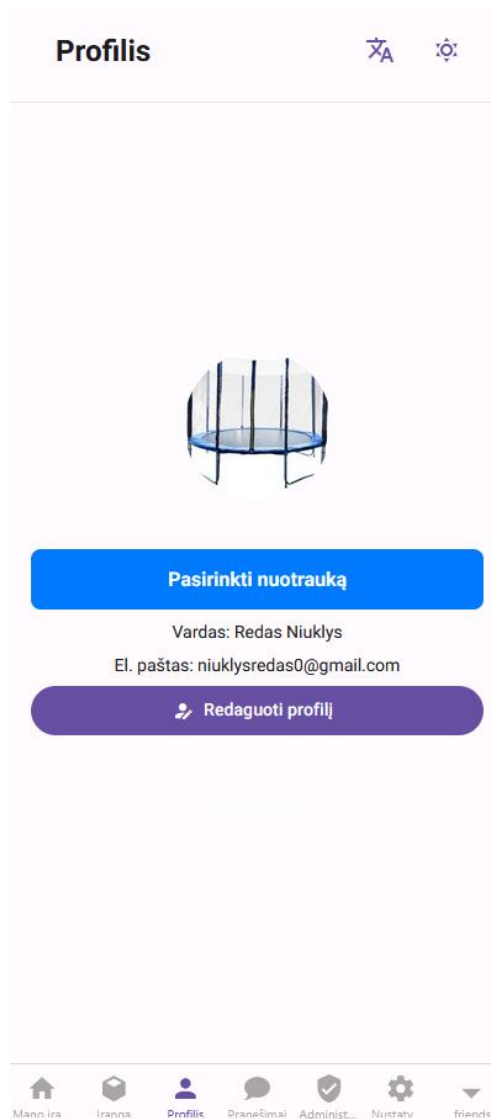
**pav. 23** Atsiliepimų modalinis langas

Iš pagrindinio lango arba Kitų įrangos lango galima pasirinkti viršutinį mygtuką „Rodyti žemėlapi“. Atidaromas Google Žemėlapis su visomis įrangomis pažymėtomis ir apačioje surašyta sąrašu, kokia įranga, kategorija, adresas bei atstumas tiesia linija nuo esamos pozicijos. (žr. **pav. 24**). Svarbus pastebėjimas – norint, kad žemėlapis rodytų tikslų atstumą reikalinga patvirtinti vietovės sekimą šiai aplikacijai. Priešingu atveju – atstumai bus nurodomi pagal Studentų g. 50, Kaunas kaip bazinis taškas.



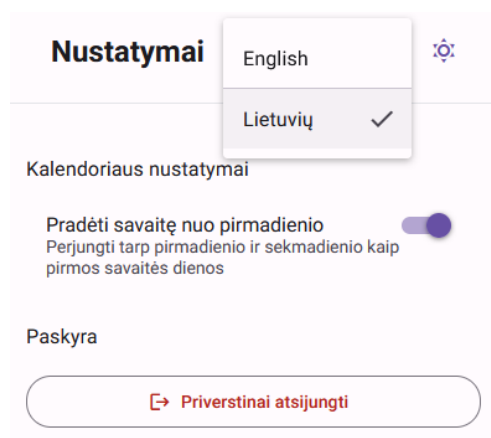
**pav. 24** Įrangų žemėlapis su atstumais

Vienas iš pagrindinių skirtukų yra skirtas profiliui, čia matoma bendrinė informacija apie paskyrą – pilnas vardas, el. Paštas bei profilio nuotrauka. Su galimybe redaguoti tai (žr. **pav. 25**)



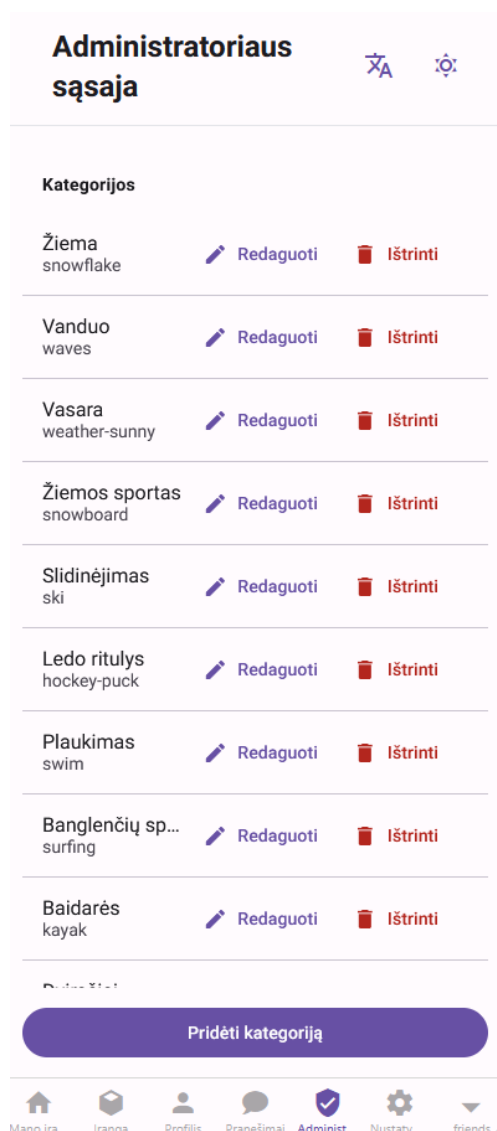
**pav. 25** Profilio langas

Kitas skirtukas – parametrai. Iš čia naudotojas gali pasirinkti, kaip rodyti kalendorių pagal savas praktikas bei čia yra priverstinis atsijungimas, nelaukiant pasibaigti žetonui. (žr. **pav. 26**). Taip pat paveikslėlyje nurodomas, kaip veikia kalbos pasikeitimas – paprastas pasirinkimas iš antraštės meniu.



**pav. 26** Nustatymų langas su kalbos pasirinkimu

Jeigu esamas naudotojas yra administratorius – jam bus matoma „Administratorius“ skirtukas. Jame – kategorijų pridėjimas, redagavimas, ištrinimas (žr. **pav. 27**)

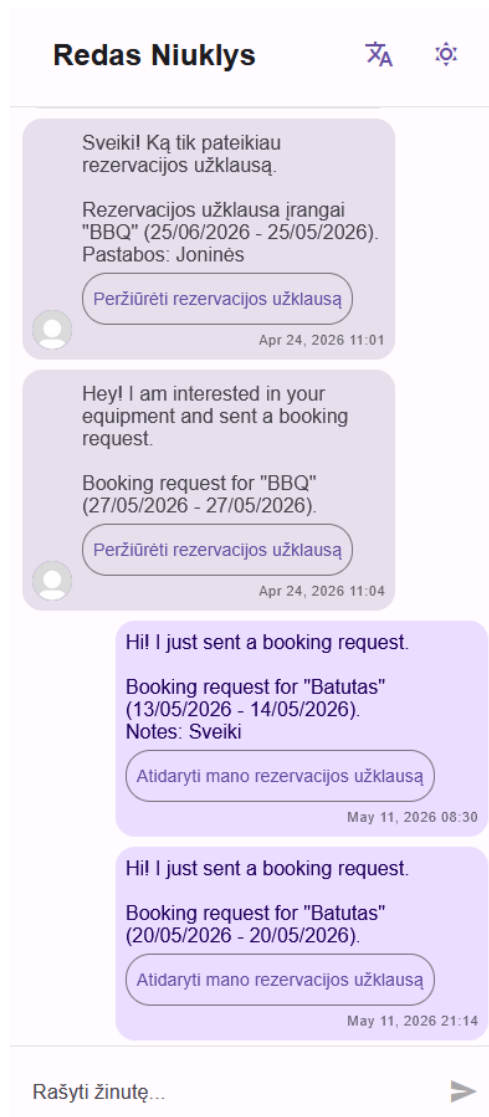


**pav. 27** Administratoriaus sąsaja

Vidurinis skirtukas navigaciniame meniu – pokalbiai. Čia matomi visi esantys pokalbiai su vartotojais. Matoma paskutinė žinutė bei jos išsiuntimo laikas (žr. **pav. 28**) atsidarius vieną iš pokalbių matoma visa istorija pilnumoje. Pavyzdyje taip pat matoma žinutės, kurios buvo sukurtos automatiškai su rezervacija. Iš pokalbio yra nuoroda tiesiai į įrangos rezervacijų sąrašą (žr. **pav. 29**)



**pav. 28** Pokalbių sąrašas



**pav. 29** Pokalbio langas

### 4.3. Diegimo vadovas

Platformos diegimo vadove nurodyta reikalingi kintamieji bei serverinių debesijos komponentų nustatymai.

Serverinėje dalyje reikalinga atlikti 3 skirtingus veiksmus:

- Paleisti AWS servigus serverio palaikymui:
  - AWS RDS duomenų bazė. Reikalinga tik paleisti. Sistema sukels pirminį planą naudojant migravimus
  - AWS ElasticBeanstalk (5). Projekte yra skriptas deploy-eb.ps1. Vienintelis skirtumas ką reikia pakeisti tai pirmose 5 eilutėse – pakeisti pavadinimus. Naudojamas .Net on Linux
  - AWS S3 – tik sukurti ir priskirti atitinkamą pavadinimą. Rekomendacija būtų leisti tik iš ElasticBeanstalk VPC leisti komunikaciją – išoriniai vartotojai neprikeltų failų
  - AWS Cloudfront – reikalinga išėjimui į HTTPS. Privalomas jeigu norima naudoti OAuth autorizavimą.
- Naudotojo sąsajos globalūs kintamieji, jei pildomi .env faile

- Užsiregistruoti Firebase platformoje. Bazinio plano yra pakankama. Iš visų servisų paketų naudosime tik *Authentication*. Firebase taip pat reikalinga susikurti serviso paskyrą. Ši paskyra yra pridėjama į failą `.env` `PUBLIC_FIREBASE_API_KEY` ir `EXPO_PUBLIC_FIREBASE_PROJECT_ID`
- Serverinės dalies Url `.env` faile : `EXPO_PUBLIC_API_ORIGIN`
- Reikalinga Google API raktas dėl Vietovės tikrinimo bei žemėlapių: `EXPO_PUBLIC_GOOGLE_MAPS_API_KEY`
- Serverinės dalies globalūs kintamieji, pildomi pačiame AWS ElasticBeanstalk per nustatymus *Environmental Variables*:
  - `ConnectionStrings` – duomenų bazės nuoroda iš AWS.
  - `Storage: Provider, S3BucketName, S3Region, S3KeyPrefix, S3PublicBaseUrl` – S3 kibiro reikšmės. `KeyPrefix` numatytas kaip uploads.
  - Jeigu naudojama – Google OAuth Client ID ir secret (1)
  - Jeigu naudojama – Facebook OAuth Client ID ir secret
  - Jeigu naudojama – Microsoft Oauth Client ID, TenantID ir secret

Supildžius šiuos globalius kintamuosius bei paleidus AWS serverį. Sistema paruošta naudojimui, mobiliąją aplikaciją teks perkurti, jog būtų užkrautos naujausia informacija, norint pasileisti naudotojo sąsają, kaip asmeninę.

#### 4.4. Administravimo vadovas

Jeigu nėra nurodomas AWS parametrai – tuomet pasikeičia diegimo sritis į lokalią. Duomenys saugomi SQLite duomenų bazėje. O nuotraukos saugomos toje pačioje direktorijoje kaip ir projektas.

Tokiu atveju administratorius turi galimybę tiesiog paleisti 2 skirtingus terminalus DEV naudotojo sąsajos versijai, kurią galima pasiekti be programėlės, o su Expo GO aplikacija, o serveris taip pat paleidžiamas komandine eilute.

Komandos paleidimui atskiruose terminaluose:

- Naudotojo sąsaja: `npm start`
- Serverio dalis: `dotnet run`



## 5. DI naudojimo savirefleksija

Šio projekto metu buvo naudojamas DI. 2 pagrindiniai tiekėjai buvo ChatGPT (OpenAI) ir Github CoPilot (skirtingi modeliai).

Šie įrankiai buvo naudojami 2 pagrindinėse srityse: analizė bei programavimas.

Analizės srityje naudojamas buvo konkurentų analizės etape, kadangi Lietuvoje tokios platformos su tokiais reikalavimais dabar neturime, pasitelktas DI atrasti kitų šalių/tiekėjų sprendimus bei įvertinti jų galimybes bei specifikacijas. Iš šių pastebėjimų bei toliau asmeninio testavimo DI atrastų sprendimų buvo pildomi reikalavimai ar praplėtimai sukurtiems funkcionalumams.

Programavimo srityje naudojamas Github CoPilot buvo naudojamas sistemos stiliaus išpildyme, kadangi nebuvo imami jokie esami šablonai. Tai padėjo išvengti galvos skausmo dėl lygiavimų, centravimų, rezoliucijos pritaikymo bei vientisumo spalvų gamoje.

Atsižvelgiant į panaudojimą – dirbtinis intelektas sutrumpino paieškos laiką analizės fazėje, kadangi vienu užklausimu yra ieškoma per daugelį šaltinių. Kas galėtų trukti kelias valandas ieškant informacijos yra pateikiama 1 pagrindiniu klausimu ir ,galbūt, 2-3 pataisymo sakiniais nukreipiant/išgryninant norimą paieškos kriterijų. Įrankis taip pat leidžia paprastai palyginti kelias svetaines/sprendimus pagal pateiktus kriterijus arba esminius skirtumus tarp jų. Programavime – leidžia užpildyti spragas esamose žinose ar sistemos stiliaus suvienodinime. DI panaudojimas šiuo atveju atitiko šablono naudą tik su papildoma pagalba atskirti komponentus į atitinkamas vietas bei dinamiškai prisitaikyti prie pridedamų skilčių.

## Rezultatai ir išvados

Atlikto projekto metu buvo gautos šios išvados ir pasiekti šie rezultatai:

1. Konkurentų analizės metu, nustatyta, jog esamos sistemos/alternatyvos suteikia tik komunikacijos galimybę tiesiogiai su savininku. Jos skirtos plačiajai auditorijai. Kurtas įrankis sprendžia problemą bei susistemizuoja rezervacijos procesą, leidžiantį įmonės/draugų rato asmenims lengviau ir su geresniu matomumu skolintis įrangą.
2. Sukurta sistema suprojektuota akcentuojant rezervacijos galimybę, paprastumą bei svarbiausia matomumą kitiems naudotojams. Projektavimas remiasi galimybe plėsti aplikacijos lygmenyje į naujus įrankius bei DI.
3. Suprojektuota sistema naudotojo sąsajoje realizuota tiek naršyklėje, tiek dedikuotoje aplikacijoje, palengvinant koregavimus įrangos nuotraukoms. Palengvinti prisijungimą prie aplikacijos – pridėta galimybė prisijungti naudojant išorines paskyras (Gmail, Facebook, Microsoft). Sistemos tinkamumas prisijungti išorinėmis paskyromis atidaro duris lengviau diegti sistemą į įmones, kur visi vartotojai dažniausiai turi jiems skirtą vienetinę paskyrą jungtis į įvairias sistemas. Taip platforma tampa tik dar viena papildoma aplikacija, kuri integruojasi į esamą įmonės ekosistemą.
4. Sistemos serverinė dalis patalpinta AWS debesijos paslaugose. Atskiriant duomenų bazę įprastiems duomenims ir papildoma talpykla paveikslėlių tipo failams. Naudotojo sąsajai galima aplikacija gali būti sudiegta Android/iOS įrenginiuose aplikaciją pasidalinus įmonės/draugų tarpe. Aplikacijos dalinimasis užtrikinima atskirtinumą nuo didžiosios naudotojų aibės, kadangi sistemoje prieinami visi naudotojų vardai. Debesijos paslaugos užtrikinima serverio prieinamumą su daug didesne tikimybe nei galėtų vidiniai resursai.
5. Sukurtai platformai parašyta 256 vienetų testų padengti 75% serverinės dalies kodo eilučių, testuojant teigiamus bei neigiamus atsakymus iš kvietimų. Didesnį padengimą pasiekti yra nerealistiška, kadangi metodai naudoja išorinius autorizavimo metodus, kurie neturi testinių variantų (Firebase, OAuth). Naudotojo sąsajai sukurtas naudotojo vadovas, nurodantis reikalingus veiksmus sistemoje iš savininko bei įprasto vartotojo pozicijos. Testų padengimas visiems kitiems valdikliams yra pakankamas, kadangi OAuth ir Firebase yra išorinės sistemos, kurios jau iš savęs užtrikinimą patikimumą bei duomenų validumą. Platforma paremta skirtukų ir modalių principu, todėl nesijaučia perkrauta ir tarp langų yra natūrali progresija. Iš įrangos yra pasiekiami visi aspektai, kurių gali reikėti įrangai ir visiems susijusiems elementams.

### Neišpildyti funkcionalumai:

1. Praplėsti rezervacijų paėmimo ir atidavimo logiką bei pririšimas prie realaus gyvenimo. Ar DI panaudojimas daiktų atpažinimui ar generuojami QR kodai nuskaityti
2. Bendros daiktų laikymo patalpos su virtualiu sandėliuko žemėlapiu (žinoma daikto pozicija iš platformos. Veikia per QR)
3. Žymių pridėjimas platesniam filtravimui ir automatinis priskyrimas pagal pagrindinę nuotrauką, išnaudojant DI.

## Literatūros sąrašas

1. Features. *Stuffly*. [Tinkle] Stuffly. [Cituota: 2026 m. 05 12 d.] <https://www.stuffly.app/en#features>.
2. EAS Build. *Expo Docs*. [Tinkle] Expo, 2026 m. 03 01 d. [Cituota: 2026 m. 03 28 d.] <https://docs.expo.dev/build/introduction/>.
3. State Machine Diagram. *NoMagic*. [Tinkle] Catia. [Cituota: 2026 m. 05 12 d.] <https://docs.nomagic.com/spaces/MD2024xR3/pages/227149100/State+Machine+diagram>.
4. Using-Moq-A-Simple-Guide. *CodeMag*. [Tinkle] 2025 m. 12 26 d. [Cituota: 2026 m. 04 20 d.] <https://www.codemag.com/Article/2305041/Using-Moq-A-Simple-Guide-to-Mocking-for-.NET>.
5. Getting Started. *AWS*. [Tinkle] Amazon. [Cituota: 2026 m. 04 26 d.] <https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/GettingStarted.html>.